

STAR RX72N - rx_drv8263 Library

1.0.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 File Index	3
2.1 File List	3
3 Directory Documentation	5
3.1 libs/rx_drv8263/inc Directory Reference	5
3.2 libs Directory Reference	5
3.3 libs/rx_drv8263 Directory Reference	6
3.4 libs/rx_drv8263/src Directory Reference	6
4 File Documentation	7
4.1 libs/rx_drv8263/inc/rx_drv8263.h File Reference	7
4.1.1 Detailed Description	8
4.1.1.1 Overview	9
4.1.1.2 Hardware Interface	9
4.1.1.3 Current Sensing	9
4.1.2 Data Structure Documentation	11
4.1.2.1 struct rx_drv8263_config_t	11
4.1.2.2 struct rx_drv8263_handle_t	12
4.1.3 Enumeration Type Documentation	13
4.1.3.1 rx_drv8263_ipropi_t	13
4.1.3.2 rx_drv8263_olp_pattern_count_t	14
4.1.3.3 rx_drv8263_olp_result_t	15
4.1.3.4 rx_drv8263_timing_us_t	16
4.1.4 Function Documentation	17
4.1.4.1 rx_drv8263_adc_to_amps()	17
4.1.4.2 rx_drv8263_clear_latched_fault()	18
4.1.4.3 rx_drv8263_init()	20
4.1.4.4 rx_drv8263_run_olp()	23
4.1.4.5 rx_drv8263_set_drvoff()	27
4.1.4.6 rx_drv8263_set_olp_boot_enable()	29
4.1.4.7 rx_drv8263_set_olp_fault_enable()	30
4.2 rx_drv8263.h	32
4.3 libs/rx_drv8263/src/rx_drv8263.c File Reference	39
4.3.1 Detailed Description	40
4.3.2 Enumeration Type Documentation	42
4.3.2.1 drv8263_delay_ticks_t	42
4.3.2.2 gpio_limits_t	42
4.3.2.3 olp_nfault_idx_t	43
4.3.3 Function Documentation	44
4.3.3.1 internal_delay_us()	44
4.3.3.2 internal_gpio_read()	45

4.3.3.3 internal_gpio_write()	46
4.3.3.4 internal_olp_apply_patterns()	47
4.3.3.5 internal_olp_decode_results()	49
4.3.3.6 internal_validate_config()	50
4.3.3.7 internal_validate_gpio()	52
4.3.3.8 rx_drv8263_clear_latched_fault()	53
4.3.3.9 rx_drv8263_init()	54
4.3.3.10 rx_drv8263_run_olp()	56
4.3.3.11 rx_drv8263_set_drvoff()	59
4.3.3.12 rx_drv8263_set_olp_boot_enable()	60
4.3.3.13 rx_drv8263_set_olp_fault_enable()	61
4.3.4 Variable Documentation	62
4.3.4.1 s_tag	62
4.4 rx_drv8263.c	62

Chapter 1

Directory Hierarchy

1.1 Directories

inc	5
rx_drv8263.h	7
libs	5
rx_drv8263	6
inc	5
rx_drv8263.h	7
src	6
rx_drv8263.c	39
rx_drv8263	6
inc	5
rx_drv8263.h	7
src	6
rx_drv8263.c	39
src	6
rx_drv8263.c	39

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

libs/rx_drv8263/inc/ rx_drv8263.h	
DRV8263H-Q1 Motor Driver Chip-Level Control API	7
libs/rx_drv8263/src/ rx_drv8263.c	
DRV8263H-Q1 Motor Driver Chip-Level Control Implementation	39

Chapter 3

Directory Documentation

3.1 libs/rx_drv8263/inc Directory Reference

Directory dependency graph for inc:

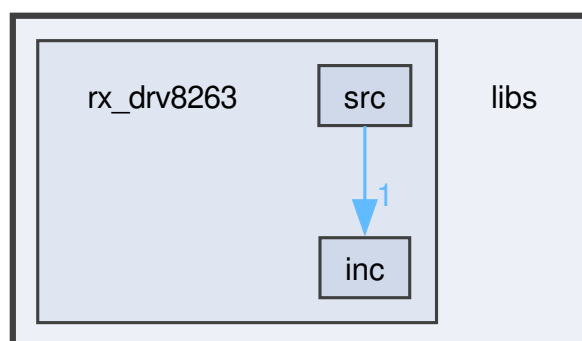


Files

- file [rx_drv8263.h](#)
DRV8263H-Q1 Motor Driver Chip-Level Control API.

3.2 libs Directory Reference

Directory dependency graph for libs:

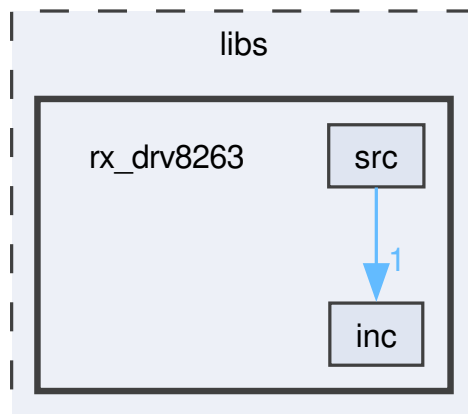


Directories

- directory [rx_drv8263](#)

3.3 libs/rx_drv8263 Directory Reference

Directory dependency graph for rx_drv8263:

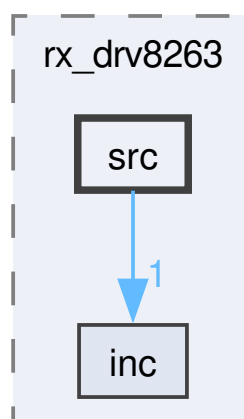


Directories

- directory [inc](#)
- directory [src](#)

3.4 libs/rx_drv8263/src Directory Reference

Directory dependency graph for src:



Files

- file [rx_drv8263.c](#)
DRV8263H-Q1 Motor Driver Chip-Level Control Implementation.

Chapter 4

File Documentation

4.1 libs/rx_drv8263/inc/rx_drv8263.h File Reference

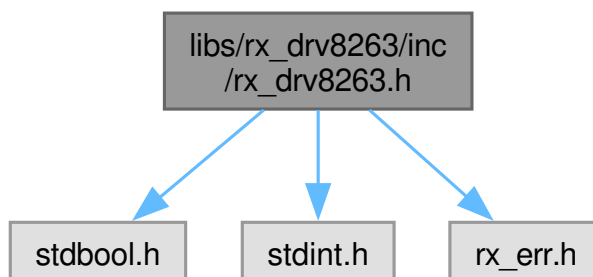
DRV8263H-Q1 Motor Driver Chip-Level Control API.

```
#include <stdbool.h>
```

```
#include <stdint.h>
```

```
#include "rx_err.h"
```

Include dependency graph for rx_drv8263.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [rx_drv8263_config_t](#)
Configuration for a DRV8263H-Q1 motor driver instance.
- struct [rx_drv8263_handle_t](#)
Runtime handle for a DRV8263H-Q1 motor driver instance.

Enumerations

- enum [rx_drv8263_ipropi_t](#): uint16_t { [k_drv8263_ipropi_mirror_ua_per_a](#), [k_drv8263_ipropi_sense_ohm](#) }
DRV8263H-Q1 IPROPI current sensing constants.
- enum [rx_drv8263_timing_us_t](#): uint16_t { [k_drv8263_nsleep_pulse_us](#), [k_drv8263_twake_us](#), [k_drv8263_olp_settle_us](#) = 50 }
DRV8263H-Q1 timing constants in microseconds.
- enum [rx_drv8263_olp_pattern_count_t](#): uint8_t { [k_drv8263_olp_pattern_count](#) = 3 }
Number of OLP test patterns.
- enum [rx_drv8263_olp_result_t](#): uint8_t { [k_drv8263_olp_normal](#) = 0, [k_drv8263_olp_open_load](#) = 1, [k_drv8263_olp_short_to_gnd](#) = 2, [k_drv8263_olp_short_to_vm](#) = 3, [k_drv8263_olp_unknown](#) = 4 }
Open Load Protection diagnostic result per motor channel.

Functions

- rx_err_t [rx_drv8263_init](#) ([rx_drv8263_handle_t](#) *handle, const [rx_drv8263_config_t](#) *config)
Initialize DRV8263H-Q1 driver instance.
- rx_err_t [rx_drv8263_set_drvoff](#) ([rx_drv8263_handle_t](#) *handle, bool active)
Control DRVOFF output disable signal.
- rx_err_t [rx_drv8263_clear_latched_fault](#) ([rx_drv8263_handle_t](#) *handle)
Clear latched fault by pulsing nSLEEP LOW.
- rx_err_t [rx_drv8263_run_olp](#) ([rx_drv8263_handle_t](#) *handle, [rx_drv8263_olp_result_t](#) *result_out1, [rx_drv8263_olp_result_t](#) *result_out2)
Run Open Load Protection diagnostic.
- static float [rx_drv8263_adc_to_amps](#) (float adc_voltage_v)
Convert IPROPI ADC voltage to motor current in amperes.
- rx_err_t [rx_drv8263_set_olp_boot_enable](#) ([rx_drv8263_handle_t](#) *handle, bool enable)
Enable or disable OLP diagnostic at boot time.
- rx_err_t [rx_drv8263_set_olp_fault_enable](#) ([rx_drv8263_handle_t](#) *handle, bool enable)
Enable or disable OLP diagnostic in fault handler.

4.1.1 Detailed Description

DRV8263H-Q1 Motor Driver Chip-Level Control API.

4.1.1.1 Overview

Provides chip-level control for the DRV8263H-Q1 H-bridge motor driver including:

- DRVOFF output disable control
- Latched fault reset via nSLEEP pulse
- Open Load Protection (OLP) diagnostics
- Current sensing ADC-to-ampere conversion

This module complements rx_motor (which handles PWM/direction) by providing DRV8263H-Q1-specific features not covered by the generic motor control API.

4.1.1.2 Hardware Interface

Signal	Direction	Description
DRVOFF	MCU -> DRV	Output disable (active-high, LOW = enabled)
nSLEEP	MCU -> DRV	Sleep control (active-low, HIGH = awake)
nFAULT	DRV -> MCU	Fault indicator (active-low, LOW = fault)
IN1	MCU -> DRV	Half-bridge B control (PWM or GPIO for OLP)
IN2	MCU -> DRV	Half-bridge A control (PWM or GPIO for OLP)

4.1.1.3 Current Sensing

DRV8263H-Q1 IPROPI output characteristics:

- Current mirror ratio: 202 uA/A
- External sense resistor: 5100 ohm (5.1k)
- $V_IPROPI = I_motor * 202e-6 * 5100 = I_motor * 1.0302$
- $I_motor = V_IPROPI / 1.0302$

Module Dependencies:

- rx_err.h - Error code definitions
- rx_log.h - Structured logging
- rx_port_utils.h - GPIO register access

NASA Power of 10 Compliance:

Rule	Status	Details
Rule 1	↔ [PASS]↔	No goto, setjmp, or recursion
Rule 2	↔ [PASS]↔	All loops bounded by enum constants
Rule 3	↔ [PASS]↔	Zero dynamic allocation
Rule 4	↔ [PASS]↔	All functions under 60 lines
Rule 5	↔ [PASS]↔	Minimum 2 validations per function
Rule 6	↔ [PASS]↔	Variables at smallest scope
Rule 7	↔ [PASS]↔	All return values checked
Rule 8	↔ [PASS]↔	C23 typed enums, minimal macros
Rule 9	↔ [WARN]↔	No function pointers in this module
Rule 10	↔ [PASS]↔	-Wall -Wextra -Werror clean

See also

`rx_motor.h` Motor PWM control (complementary to this module)

`hardware_config.h` Pin assignments for DRVOFF, nSLEEP, nFAULT

Author

Locked, Inc.

Date

2026-03-03

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Version

1.0.0

Definition in file [rx_drv8263.h](#).

4.1.2 Data Structure Documentation

4.1.2.1 struct rx_drv8263_config_t

Configuration for a DRV8263H-Q1 motor driver instance.

Specifies GPIO port/pin assignments for each control signal and OLP diagnostic behavior flags. Port numbers use rx_port_utils.h constants (e.g., k_rx_port_6 = 6, k_rx_port_e = 14).

Usage Example:

```
const rx_drv8263_config_t config = {
    .port_drvoff = 6, .pin_drvoff = 1, // P61
    .port_nsleep = 6, .pin_nsleep = 0, // P60
    .port_nfault = 1, .pin_nfault = 5, // P15
    .port_in1 = 1, .pin_in1 = 7, // P17
    .port_in2 = 2, .pin_in2 = 3, // P23
    .olp_enable_boot = true,
    .olp_enable_fault = true,
};
```

Invariant

port_drvoff, port_nsleep, port_nfault, port_in1, port_in2 must be valid RX72N ports
pin values must be 0-7 (8-bit port width)

See also

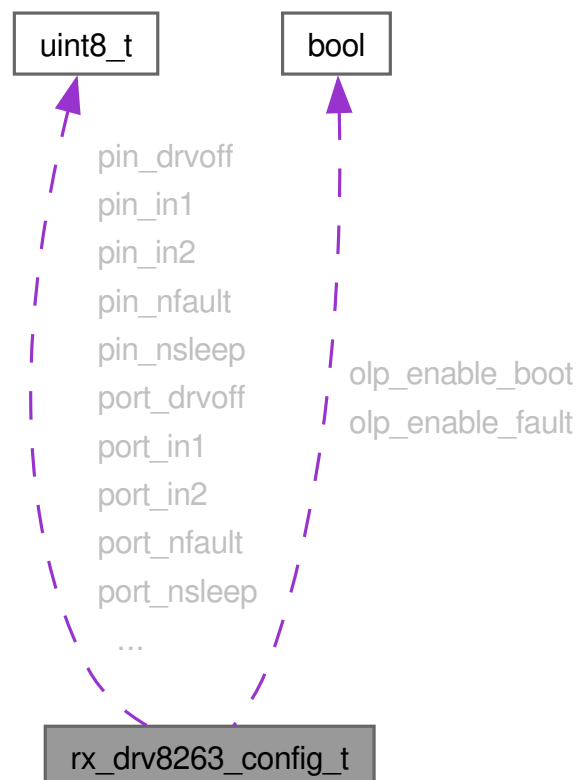
[rx_drv8263_init\(\)](#) Uses this configuration

Since

Version 1.0.0

Definition at line 235 of file [rx_drv8263.h](#).

Collaboration diagram for rx_drv8263_config_t:



Data Fields

bool	olp_enable_boot	Run OLP diagnostic during rx_drv8263_init()
bool	olp_enable_fault	Run OLP diagnostic after clearing latched faults
uint8_t	pin_drvooff	DRVOFF GPIO pin number within port (0-7)
uint8_t	pin_in1	IN1 GPIO pin number within port (0-7)
uint8_t	pin_in2	IN2 GPIO pin number within port (0-7)
uint8_t	pin_nfault	nFAULT GPIO pin number within port (0-7)
uint8_t	pin_nsleep	nSLEEP GPIO pin number within port (0-7)
uint8_t	port_drvooff	DRVOFF GPIO port number (e.g., 6 for PORT6)
uint8_t	port_in1	IN1 GPIO port number (for OLP pattern output)
uint8_t	port_in2	IN2 GPIO port number (for OLP pattern output)
uint8_t	port_nfault	nFAULT GPIO port number (input, directly read for OLP)
uint8_t	port_nsleep	nSLEEP GPIO port number

4.1.2.2 struct rx_drv8263_handle_t

Runtime handle for a DRV8263H-Q1 motor driver instance.

Stores configuration and initialization state. Allocated by caller (typically static), initialized by [rx_drv8263_init\(\)](#).

Invariant

initialized is false until [rx_drv8263_init\(\)](#) succeeds

```
rx_drv8263_handle_t drv = {};
rx_err_t err = rx_drv8263_init(&drv, &config);
if (drv.initialized) { // ready to use }
```

See also

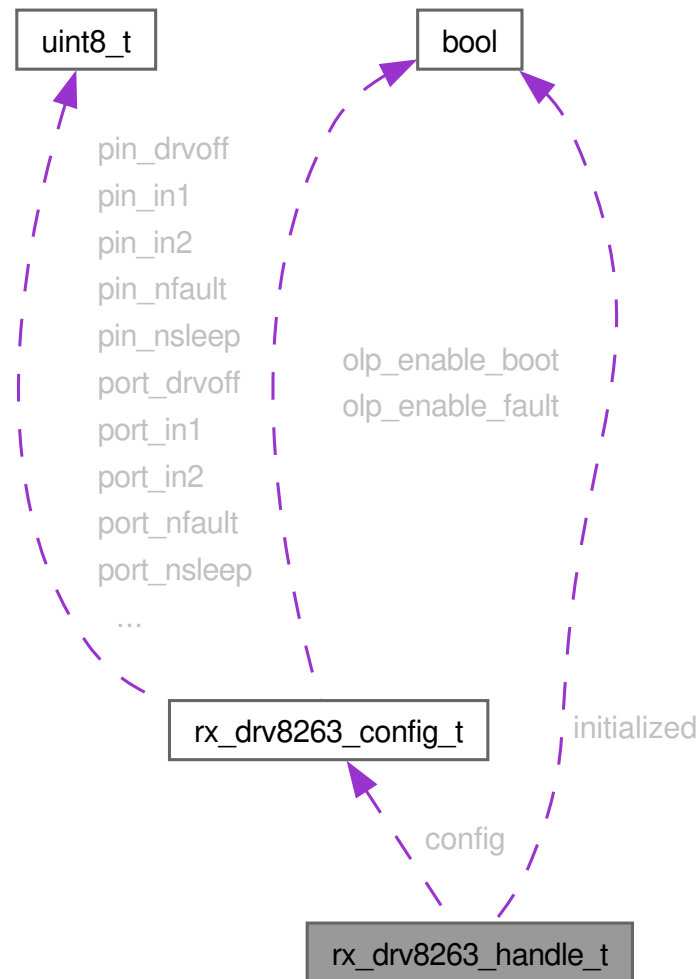
[rx_drv8263_init\(\)](#) Initializes handle

Since

Version 1.0.0

Definition at line 269 of file [rx_drv8263.h](#).

Collaboration diagram for rx_drv8263_handle_t:



Data Fields

rx_drv8263_config_t	config	Copied configuration (OLP flags mutable via setters)
bool	initialized	true after successful init, false otherwise

4.1.3 Enumeration Type Documentation

4.1.3.1 rx_drv8263_ipropi_t

```
enum rx_drv8263_ipropi_t : uint16_t
```

DRV8263H-Q1 IPROPI current sensing constants.

Constants for converting IPROPI analog voltage to motor current. $V_IPROPI = I_motor * k_drv8263_ipropi_mirror_ua_per_a * k_drv8263_ipropi_sense_ohm * 1e-6$
 $I_motor = V_IPROPI / 1.0302$

See also

[rx_drv8263_adc_to_amps\(\)](#) Conversion helper function

```
float divisor = (float)k_drv8263_ipropi_mirror_ua_per_a
                * (float)k_drv8263_ipropi_sense_ohm * 1e-6F;
// divisor == 1.0302F
```

Invariant

```
k_drv8263_ipropi_mirror_ua_per_a == 202 (integer compile-time constant)
k_drv8263_ipropi_sense_ohm == 5100 (integer compile-time constant)
```

Since

Version 1.0.0

Enumerator

k_drv8263_ipropi_mirror_ua_per_a	Current mirror ratio: 202 uA per amp of motor current
k_drv8263_ipropi_sense_ohm	IPROPI sense resistor value in ohms (5.1k, 1% tolerance)

Definition at line 101 of file [rx_drv8263.h](#).

```
00101      : uint16_t {
00102  k_drv8263_ipropi_mirror_ua_per_a =
00103      202, /**< Current mirror ratio: 202 uA per amp of motor current */
00104  k_drv8263_ipropi_sense_ohm =
00105      5100, /**< IPROPI sense resistor value in ohms (5.1k, 1% tolerance) */
00106 } rx_drv8263_ipropi_t;
```

4.1.3.2 rx_drv8263_olp_pattern_count_t

```
enum rx_drv8263_olp_pattern_count_t : uint8_t
```

Number of OLP test patterns.

The DRV8263H-Q1 OLP diagnostic uses exactly 3 test patterns (count=3). Each pattern sets a unique IN1/IN2 combination while DRVOFF=HIGH, then reads nFAULT to determine load condition:

- Pattern 0: IN1=0, IN2=0 (both inputs low, baseline reading)
- Pattern 1: IN1=1, IN2=0 (drives OUT1 side, detects open on OUT1)
- Pattern 2: IN1=0, IN2=1 (drives OUT2 side, detects open on OUT2)

The 3-bit nFAULT result vector from these patterns is decoded against the OLP truth table to classify the load as normal, open, shorted to GND, or shorted to VM.

See also

[rx_drv8263_run_olp\(\)](#) Executes the OLP diagnostic using these patterns
[rx_drv8263_olp_result_t](#) Result classification from pattern decoding

```
for (uint8_t i = 0; i < k_drv8263_olp_pattern_count; i++) {
    // apply pattern[i], read nFAULT
}
```

Invariant

```
k_drv8263_olp_pattern_count == 3
```

Since

Version 1.0.0

Enumerator

<code>k_drv8263_olp_pattern_count</code>	Three test patterns for OLP diagnostic
--	--

Definition at line 162 of file `rx_drv8263.h`.

```
00162         : uint8_t {
00163     k_drv8263_olp_pattern_count = 3, /**< Three test patterns for OLP diagnostic */
00164 } rx_drv8263_olp_pattern_count_t;
```

4.1.3.3 rx_drv8263_olp_result_t

enum `rx_drv8263_olp_result_t` : `uint8_t`

Open Load Protection diagnostic result per motor channel.

Results from the DRV8263H-Q1 OLP diagnostic sequence. The 3-pattern test with DRVOFF=HIGH reads nFAULT for each pattern to determine load condition.

OLP Truth Table (nFAULT readings for patterns {IN1=0/IN2=0, IN1=1/IN2=0, IN1=0/IN2=1}):

- {1,1,1} = Normal (no fault)
- {1,1,0} = Open load on OUT2
- {1,0,1} = Open load on OUT1
- {0,1,1} = Short to GND
- {0,0,0} = Short to VM (or both shorted)

See also

`rx_drv8263_run_olp()` Executes OLP diagnostic

```
rx_drv8263_olp_result_t res = k_drv8263_olp_unknown;
// ... run OLP ...
if (res != k_drv8263_olp_normal) {
    // handle fault condition
}
```

Invariant

Each value maps uniquely to a diagnostic state (0=normal, 1=open, 2=gnd, 3=vm, 4=unknown)

Since

Version 1.0.0

Enumerator

<code>k_drv8263_olp_normal</code>	No fault detected, load connected normally
-----------------------------------	--

k_drv8263_olp_open_load	Open load detected (motor disconnected)
k_drv8263_olp_short_to_gnd	Short circuit to ground detected
k_drv8263_olp_short_to_vm	Short circuit to motor supply (VM) detected
k_drv8263_olp_unknown	Unknown or ambiguous diagnostic result

Definition at line 199 of file [rx_drv8263.h](#).

```
00199      : uint8_t {
00200  k_drv8263_olp_normal      = 0, /**< No fault detected, load connected normally */
00201  k_drv8263_olp_open_load  = 1, /**< Open load detected (motor disconnected) */
00202  k_drv8263_olp_short_to_gnd = 2, /**< Short circuit to ground detected */
00203  k_drv8263_olp_short_to_vm = 3, /**< Short circuit to motor supply (VM) detected */
00204  k_drv8263_olp_unknown    = 4, /**< Unknown or ambiguous diagnostic result */
00205 } rx_drv8263_olp_result_t;
```

4.1.3.4 rx_drv8263_timing_us_t

```
enum rx_drv8263_timing_us_t : uint16_t
```

DRV8263H-Q1 timing constants in microseconds.

Timing parameters from DRV8263H-Q1 datasheet for fault reset and OLP.

Invariant

k_drv8263_nsleep_pulse_us must be in range [5, 35] per datasheet

```
uint16_t pulse = k_drv8263_nsleep_pulse_us; // 20 us
uint16_t wake  = k_drv8263_twake_us;        // 1200 us
uint16_t settle = k_drv8263_olp_settle_us;   // 50 us
```

Since

Version 1.0.0

Enumerator

k_drv8263_nsleep_pulse_us	nSLEEP LOW pulse duration for latched fault reset (target middle of 5-35 us window)
k_drv8263_twake_us	Wake-up time after nSLEEP HIGH (nFAULT returns HIGH, ~1.2 ms typ)
k_drv8263_olp_settle_us	Settling time between OLP test patterns (conservative)

Definition at line 125 of file [rx_drv8263.h](#).

```
00125      : uint16_t {
00126  k_drv8263_nsleep_pulse_us =
00127      20, /**< nSLEEP LOW pulse duration for latched fault reset (target middle of 5-35 us window) */
00128  k_drv8263_twake_us =
00129      1200, /**< Wake-up time after nSLEEP HIGH (nFAULT returns HIGH, ~1.2 ms typ) */
00130  k_drv8263_olp_settle_us = 50, /**< Settling time between OLP test patterns (conservative) */
00131 } rx_drv8263_timing_us_t;
```

4.1.4 Function Documentation

4.1.4.1 rx_drv8263_adc_to_amps()

```
float rx_drv8263_adc_to_amps (
    float adc_voltage_v)  [inline], [static]
```

Convert IPROPI ADC voltage to motor current in amperes.

Pure math conversion using DRV8263H-Q1 current sensing parameters: $I_{\text{motor}} = V_{\text{IPROPI}} / 1.0302$
 where $1.0302 = 202\text{e-}6 \text{ A/A} * 5100 \text{ ohm}$

Parameters

in	adc_voltage↔ _v	IPROPI analog voltage in volts (0.0 to 3.3V)
----	--------------------	--

Returns

Motor current in amperes

Precondition

adc_voltage_v must be non-negative

adc_voltage_v should be within ADC range (0.0 to 3.3V typical)

Postcondition

Return value ≥ 0.0

Return value is $\text{adc_voltage_v} / \text{k_ipropi_divisor}$ (exact mathematical relationship)

Note

Pure function, thread-safe, no side effects

Example:

```
float voltage = adc_read_voltage(k_adc_ch_motor0_current);
float amps = rx_drv8263_adc_to_amps(voltage);
// 1.65V -> 1.602A
```

Since

Version 1.0.0

Definition at line 446 of file [rx_drv8263.h](#).

```
00447 {
00448     /* Combined IPROPI conversion factor: 202e-6 * 5100 = 1.0302 */
00449     static const float s_ipropi_divisor = 1.0302F;
00450
00451     return adc_voltage_v / s_ipropi_divisor;
00452 }
```

4.1.4.2 rx_drv8263_clear_latched_fault()

```
rx_err_t rx_drv8263_clear_latched_fault (
    rx_drv8263_handle_t * handle) [nodiscard]
```

Clear latched fault by pulsing nSLEEP LOW.

DRV8263H-Q1 latches faults until nSLEEP is pulsed LOW for 5-35 us. This function:

1. Drives nSLEEP LOW
2. Busy-waits ~20 us (middle of 5-35 us window)
3. Drives nSLEEP HIGH
4. Sleeps one ThreadX tick (~10 ms with 100 Hz tick) to satisfy tWAKE (~1.2 ms); coarse-grained because tx_thread_sleep granularity is one tick (nFAULT is not polled – it may be configured as a POEG input)

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.
----	--------	---

Returns

k_rx_ok on success

Return values

k_rx_err_null_ptr	if handle is nullptr
k_rx_err_invalid_state	if not initialized

Precondition

handle must be initialized via [rx_drv8263_init\(\)](#)
 Motor outputs should be stopped before clearing fault

Postcondition

Latched fault cleared (if fault condition removed)
 nSLEEP returns to HIGH (awake)

Warning

Uses CPU busy-wait for ~20 us pulse. Interrupts are NOT disabled during the pulse, so actual duration may be slightly longer.

Note

Uses tx_thread_sleep(1) for tWAKE delay (10 ms tick >= 1.2 ms required)

Since

Version 1.0.0

The DRV8263H-Q1 latches faults (overcurrent, thermal shutdown) until the nSLEEP pin is pulsed LOW for a duration within the 5-35 us datasheet window. This function executes the following sequence:

1. Drive nSLEEP LOW to begin the fault reset pulse
2. Busy-wait for `k_drv8263_nsleep_pulse_us` (~20 us), targeting the middle of the 5-35 us specification window
3. Drive nSLEEP HIGH to wake the driver
4. Wait for tWAKE using `tx_thread_sleep(k_twake_threadx_ticks)`. One ThreadX tick (~10 ms) provides sufficient margin over the ~1.2 ms tWAKE specification. After tWAKE, nFAULT returns HIGH if the underlying fault condition has been removed.

Timing Notes:

The nSLEEP pulse uses a CPU busy-wait loop (`internal_delay_us`) since the required precision (~20 us) is below ThreadX tick granularity (~10 ms). The tWAKE delay uses `tx_thread_sleep` to yield the CPU. nFAULT cannot be polled after the pulse because it may be configured as a POEG (Port Output Enable for GPT) input rather than a standard GPIO input.

Precondition

handle must be initialized via `rx_drv8263_init()`

Motor outputs should be stopped (PWM disabled) before clearing a fault to prevent unexpected motor movement when the driver re-enables

Postcondition

nSLEEP pin is HIGH (driver awake)

Latched fault is cleared if the underlying fault condition has been removed

At least `k_twake_threadx_ticks` ThreadX ticks have elapsed (tWAKE satisfied)

Note

Not thread-safe; caller must provide synchronization

Cannot verify fault clearance because nFAULT may be a POEG input

See also

`rx_drv8263_init()` Must be called before this function

`rx_drv8263_run_olp()` Run OLP after clearing fault to diagnose load condition

`k_drv8263_nsleep_pulse_us` nSLEEP pulse duration constant

`k_twake_threadx_ticks` ThreadX tick delay for tWAKE

Since

Version 1.0.0

Definition at line 658 of file [rx_drv8263.c](#).

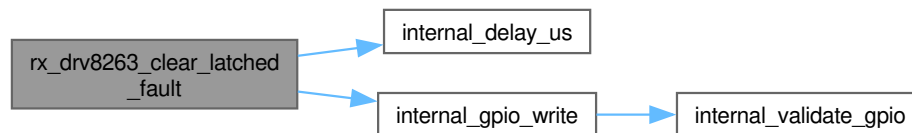
```

00659 {
00660     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00661     if (!handle->initialized) {
00662         rx_log_error(s_tag, "Not initialized");
00663         return k_rx_err_invalid_state;
00664     }
00665     /* Step 1: Drive nSLEEP LOW to begin fault reset pulse */
00666     internal_gpio_write(handle->config.port_nsleep, handle->config.pin_nsleep, false);
00667     /* Step 2: Hold LOW for ~20 us (within 5-35 us spec window) */
00668     internal_delay_us((uint32_t)k_drv8263_nsleep_pulse_us);
00669     /* Step 3: Drive nSLEEP HIGH to wake driver */
00670     internal_gpio_write(handle->config.port_nsleep, handle->config.pin_nsleep, true);
00671     /* Step 4: Wait for tWAKE (~1.2 ms). One ThreadX tick = 10 ms provides
00672      * sufficient margin. Cannot poll nFAULT as it may be configured as POEG input. */
00673     /* NASA Rule 7: capture return value; intentionally discarded (sleep cannot
00674      * meaningfully fail in this context, but we document the discard). */
00675     const UINT sleep_status = tx_thread_sleep((ULONG)k_twake_threadx_ticks);
00676     (void)sleep_status;
00677     rx_log_info(s_tag, "Latched fault cleared");
00678     return k_rx_ok;
00679 }

```

References [rx_drv8263_handle_t::config](#), [rx_drv8263_handle_t::initialized](#), [internal_delay_us\(\)](#), [internal_gpio_write\(\)](#), [k_drv8263_nsleep_pulse_us](#), [k_twake_threadx_ticks](#), [rx_drv8263_config_t::pin_nsleep](#), [rx_drv8263_config_t::port_nsleep](#), and [s_tag](#).

Here is the call graph for this function:



4.1.4.3 rx_drv8263_init()

```

rx_err_t rx_drv8263_init (
    rx_drv8263_handle_t * handle,
    const rx_drv8263_config_t * config) [nodiscard]

```

Initialize DRV8263H-Q1 driver instance.

Copies configuration, validates GPIO assignments, and optionally runs OLP diagnostic if `olp_enable_`↵ boot is set.

Parameters

out	handle	Pointer to handle structure. Must not be nullptr.
in	config	Pointer to configuration. Must not be nullptr.

Returns

k_rx_ok on success

Return values

k_rx_err_null_ptr	if handle or config is nullptr
k_rx_err_invalid_arg	if GPIO port/pin values are invalid

Precondition

handle must point to valid memory
config must contain valid GPIO port/pin assignments

Postcondition

handle->initialized = true on success
handle->config contains copy of *config

Note

Not thread-safe

Since

Version 1.0.0

Validates all GPIO assignments in the provided configuration, copies the configuration into the handle, and marks the handle as initialized. If the configuration flag `olp_enable_boot` is set, the function additionally runs a full Open Load Protection (OLP) diagnostic sequence during initialization to detect disconnected or shorted motors before the control loop begins. OLP failures at boot are logged as warnings but do not prevent initialization from succeeding.

Initialization sequence:

1. Validate handle and config pointers (null check)
2. Validate all five GPIO port/pin pairs (DRVOFF, nSLEEP, nFAULT, IN1, IN2)
3. Copy configuration into handle
4. Set handle->initialized = true
5. (Optional) Run OLP diagnostic if `olp_enable_boot` is true

Precondition

handle must point to valid, writable memory of size $\geq \text{sizeof}(\text{rx_drv8263_handle_t})$
config must contain valid GPIO port (0-16) and pin (0-7) assignments for all signals
GPIO ports referenced in config must have their clocks enabled and pin directions set

Postcondition

handle->initialized == true on success
 handle->config contains a complete copy of *config
 On failure, handle is not modified (remains in its prior state)

Note

Not thread-safe; do not call concurrently for the same handle

Example:

```
static rx_drv8263_handle_t drv;
const rx_drv8263_config_t cfg = {
    .port_drvoft = 6, .pin_drvoft = 1,
    .port_nsleap = 6, .pin_nsleap = 0,
    .port_nfault = 1, .pin_nfault = 5,
    .port_in1 = 1, .pin_in1 = 7,
    .port_in2 = 2, .pin_in2 = 3,
    .olp_enable_boot = true,
    .olp_enable_fault = true,
};
rx_err_t err = rx_drv8263_init(&drv, &cfg);
```

See also

[rx_drv8263_set_drvoft\(\)](#) Control DRVOFF after initialization
[rx_drv8263_run_olp\(\)](#) Manual OLP diagnostic
[rx_drv8263_clear_latched_fault\(\)](#) Fault reset

Since

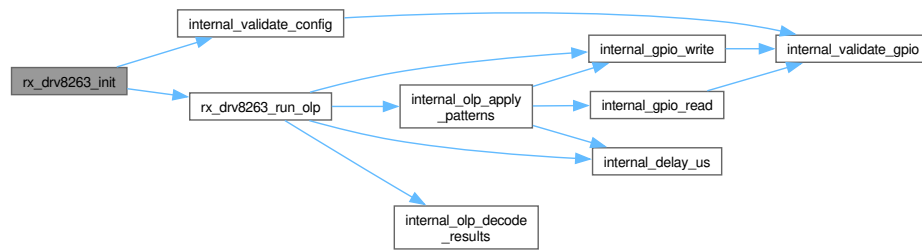
Version 1.0.0

Definition at line 539 of file [rx_drv8263.c](#).

```
00540 {
00541     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00542     RX_CHECK_NULL_PTR(config, s_tag, "config is nullptr");
00543
00544     const rx_err_t err = internal_validate_config(config);
00545     if (err != k_rx_ok) {
00546         return err;
00547     }
00548
00549     handle->config = *config;
00550     handle->initialized = true;
00551
00552     rx_log_info(s_tag, "DRV8263H-Q1 driver initialized");
00553
00554     /* Run OLP diagnostic at boot if configured */
00555     if (config->olp_enable_boot) {
00556         rx_drv8263_olp_result_t result_out1 = k_drv8263_olp_unknown;
00557         rx_drv8263_olp_result_t result_out2 = k_drv8263_olp_unknown;
00558
00559         (void)rx_drv8263_run_olp(handle, &result_out1, &result_out2);
00560         bool out1_abnormal = (bool)(result_out1 != k_drv8263_olp_normal);
00561         bool out2_abnormal = (bool)(result_out2 != k_drv8263_olp_normal);
00562         if ((bool)((int)out1_abnormal | (int)out2_abnormal)) {
00563             rx_log_warn(s_tag, "Boot OLP detected abnormal load condition");
00564         }
00565     }
00566
00567     return k_rx_ok;
00568 }
```

References [rx_drv8263_handle_t::config](#), [rx_drv8263_handle_t::initialized](#), [internal_validate_config\(\)](#), [k_drv8263_olp_normal](#), [k_drv8263_olp_unknown](#), [rx_drv8263_config_t::olp_enable_boot](#), [rx_drv8263_run_olp\(\)](#), and [s_tag](#).

Here is the call graph for this function:



4.1.4.4 rx_drv8263_run_olp()

```

rx_err_t rx_drv8263_run_olp (
    rx_drv8263_handle_t * handle,
    rx_drv8263_olp_result_t * result_out1,
    rx_drv8263_olp_result_t * result_out2) [nodiscard]

```

Run Open Load Protection diagnostic.

Executes the DRV8263H-Q1 OLP diagnostic sequence with DRVOFF=HIGH:

1. Set DRVOFF = HIGH (standby mode, enables OLP)
2. For each of 3 test patterns, set IN1/IN2 and read nFAULT:
 - Pattern 0: IN1=0, IN2=0 -> read nFAULT
 - Pattern 1: IN1=1, IN2=0 -> read nFAULT
 - Pattern 2: IN1=0, IN2=1 -> read nFAULT
3. Decode 3-bit nFAULT result against truth table
4. Set DRVOFF = LOW (return to active mode)

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.
out	result_out1	OLP result for OUT1 (IN1 side). Must not be nullptr.
out	result_out2	OLP result for OUT2 (IN2 side). Must not be nullptr.

Returns

k_rx_ok on success

Return values

k_rx_err_null_ptr	if any pointer is nullptr
k_rx_err_invalid_state	if not initialized

Precondition

handle must be initialized via `rx_drv8263_init()`

Motor must be stopped (PWM disabled) before running OLP

Postcondition

DRVOFF returns to LOW (outputs enabled)

result_out1 and result_out2 contain diagnostic results

Warning

IN1/IN2 pins are temporarily driven as GPIO during OLP. Motor PWM must be stopped before calling this function.

Note

Execution time: ~200 us (3 patterns with settling delays)

```
rx_drv8263_handle_t drv;
// ... init drv ...

// Stop PWM before running OLP
rx_drv8263_olp_result_t out1 = k_drv8263_olp_unknown;
rx_drv8263_olp_result_t out2 = k_drv8263_olp_unknown;
rx_err_t err = rx_drv8263_run_olp(&drv, &out1, &out2);
if (err == k_rx_ok) {
    if (out1 != k_drv8263_olp_normal) { // OUT1 fault
    }
    if (out2 != k_drv8263_olp_normal) { // OUT2 fault
    }
}
```

Since

Version 1.0.0

Run Open Load Protection diagnostic.

Executes the full DRV8263H-Q1 OLP diagnostic to determine the load condition on motor outputs OUT1 and OUT2. The sequence temporarily places the driver in standby mode (DRVOFF HIGH), applies three IN1/IN2 test patterns while sampling nFAULT after each, then restores active mode (DRVOFF LOW).

Full OLP test sequence:

1. Set DRVOFF = HIGH (standby mode, enables OLP detection logic)
2. Wait `k_drv8263_olp_settle_us` (~50 us) for DRVOFF to take effect
3. Apply 3 test patterns, each followed by `k_drv8263_olp_settle_us` delay:
 - Pattern 0: IN1=0, IN2=0 -> read nFAULT (baseline)
 - Pattern 1: IN1=1, IN2=0 -> read nFAULT (tests OUT1 path)
 - Pattern 2: IN1=0, IN2=1 -> read nFAULT (tests OUT2 path)
4. Return IN1 and IN2 to LOW (safe state)
5. Set DRVOFF = LOW (return to active motor control mode)
6. Decode 3-bit nFAULT result vector against the truth table

nFAULT Pattern Ordering:

Patterns are applied in index order (`k_olp_pattern_both_low`, `k_olp_pattern_in1_high`, `k_olp_pattern_in2_high`). The nFAULT reading for each pattern is stored at the corresponding array index.

OLP Truth Table:

Pattern 0	Pattern 1	Pattern 2	OUT1 Result	OUT2 Result
1	1	1	Normal	Normal
1	1	0	Normal	Open Load
1	0	1	Open Load	Normal
0	1	1	Short to GND	Short to GND
0	0	0	Short to VM	Short to VM
other	other	other	Unknown	Unknown

Precondition

handle must be initialized via [rx_drv8263_init\(\)](#)

Motor PWM must be stopped (IN1/IN2 are driven as GPIO during OLP)

nFAULT pin must be readable as a standard GPIO input (not solely POEG)

Postcondition

DRVOFF returns to LOW (outputs re-enabled)

IN1 and IN2 are both LOW (safe / coast state)

result_out1 and result_out2 contain decoded OLP diagnostic results

Note

Not thread-safe; caller must provide synchronization

Execution time: approximately 200 us total (4 x `k_drv8263_olp_settle_us` settling delays plus GPIO read/write overhead)

See also

[rx_drv8263_init\(\)](#) Calls this at boot when `olp_enable_boot` is true

[rx_drv8263_set_drvoff\(\)](#) DRVOFF control used internally

[internal_olp_apply_patterns\(\)](#) Applies the 3 test patterns

[internal_olp_decode_results\(\)](#) Decodes the nFAULT readings

[k_drv8263_olp_settle_us](#) Settling delay between patterns

Since

Version 1.0.0

Definition at line 746 of file [rx_drv8263.c](#).

```

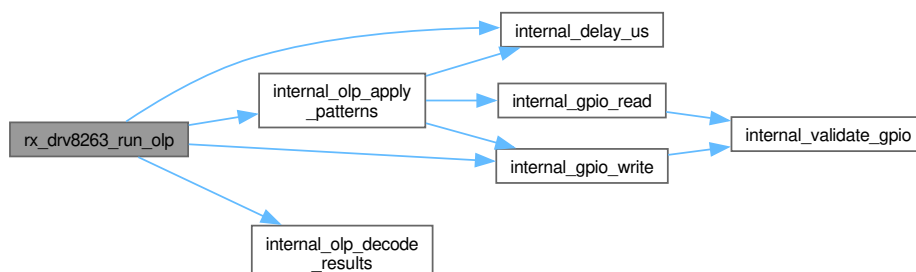
00749 {
00750     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00751     RX_CHECK_NULL_PTR(result_out1, s_tag, "result_out1 is nullptr");
00752     RX_CHECK_NULL_PTR(result_out2, s_tag, "result_out2 is nullptr");
00753
00754     if (!handle->initialized) {
00755         rx_log_error(s_tag, "Not initialized");
00756         return k_rx_err_invalid_state;
00757     }
00758
00759     /* Default results */
00760     *result_out1 = k_drv8263_olp_unknown;
00761     *result_out2 = k_drv8263_olp_unknown;
00762
00763     /* Step 1: Set DRVOFF = HIGH (standby mode, enables OLP detection) */
00764     internal_gpio_write(handle->config.port_drvoft, handle->config.pin_drvoft, true);
00765     internal_delay_us((uint32_t)k_drv8263_olp_settle_us);
00766
00767     /* Step 2: Apply 3 test patterns and read nFAULT for each */
00768     bool nfault_readings[k_drv8263_olp_pattern_count] = {false};
00769     internal_olp_apply_patterns(handle, nfault_readings);
00770
00771     /* Step 3: Return IN1/IN2 to LOW (safe state) */
00772     internal_gpio_write(handle->config.port_in1, handle->config.pin_in1, false);
00773     internal_gpio_write(handle->config.port_in2, handle->config.pin_in2, false);
00774
00775     /* Step 4: Set DRVOFF = LOW (return to active mode) */
00776     internal_gpio_write(handle->config.port_drvoft, handle->config.pin_drvoft, false);
00777
00778     /* Step 5: Decode OLP results from nFAULT readings */
00779     internal_olp_decode_results(nfault_readings[k_olp_pattern_both_low],
00780                                nfault_readings[k_olp_pattern_in1_high],
00781                                nfault_readings[k_olp_pattern_in2_high],
00782                                result_out1,
00783                                result_out2);
00784
00785     return k_rx_ok;
00786 }

```

References [rx_drv8263_handle_t::config](#), [rx_drv8263_handle_t::initialized](#), [internal_delay_us\(\)](#), [internal_gpio_write\(\)](#), [internal_olp_apply_patterns\(\)](#), [internal_olp_decode_results\(\)](#), [k_drv8263_olp_pattern_count](#), [k_drv8263_olp_settle_us](#), [k_drv8263_olp_unknown](#), [k_olp_pattern_both_low](#), [k_olp_pattern_in1_high](#), [k_olp_pattern_in2_high](#), [rx_drv8263_config_t::pin_drvoft](#), [rx_drv8263_config_t::pin_in1](#), [rx_drv8263_config_t::pin_in2](#), [rx_drv8263_config_t::port_drvoft](#), [rx_drv8263_config_t::port_in1](#), [rx_drv8263_config_t::port_in2](#), and [s_tag](#).

Referenced by [rx_drv8263_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.4.5 rx drv8263 set drvoff()

```
rx_err_t rx_drv8263_set_drvoff (
    rx_drv8263_handle_t * handle,
    bool active) [nodiscard]
```

Control DRVOFF output disable signal.

DRVOFF is active-high: setting active=true drives DRVOFF HIGH to disable H-bridge outputs. Setting active=false drives DRVOFF LOW to enable outputs.

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.
in	active	true = DRVOFF HIGH (outputs disabled), false = DRVOFF LOW (outputs enabled)

Returns

k_rx_ok on success

Return values

k_rx_err_null_ptr	if handle is nullptr
k_rx_err_invalid_state	if not initialized

Precondition

handle must be initialized via [rx_drv8263_init\(\)](#)

DRVOFF GPIO must be configured as output (hardware_init responsibility)

Postcondition

DRVOFF GPIO set to requested level

Handle remains initialized; no other H-bridge control pins modified

Note

Not thread-safe

Since

Version 1.0.0

Drives the DRVOFF GPIO to enable or disable the DRV8263H-Q1 H-bridge outputs. DRVOFF is active-high: when active=true, the pin is driven HIGH and both half-bridge outputs (OUT1, OUT2) are placed in a high-impedance state. When active=false, the pin is driven LOW and normal PWM-controlled operation resumes.

DRVOFF must be asserted HIGH before running Open Load Protection (OLP) diagnostics, and de-asserted LOW to return to normal motor operation.

Precondition

handle must be initialized via [rx_drv8263_init\(\)](#)

DRVOFF GPIO must be configured as an output pin (hardware_init responsibility)

Postcondition

DRVOFF GPIO reflects the requested level (HIGH if active, LOW otherwise)

No other GPIO pins (nSLEEP, IN1, IN2) are modified

Note

Not thread-safe; caller must provide synchronization when accessing the same handle from multiple contexts

See also

[rx_drv8263_init\(\)](#) Must be called before this function

[rx_drv8263_run_olp\(\)](#) Internally uses DRVOFF HIGH for OLP sequence

Since

Version 1.0.0

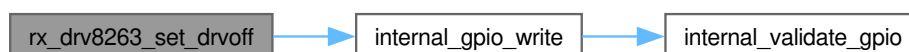
Definition at line 599 of file [rx_drv8263.c](#).

```

00600 {
00601   RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00602
00603   if (!handle->initialized) {
00604     rx_log_error(s_tag, "Not initialized");
00605     return k_rx_err_invalid_state;
00606   }
00607
00608   internal_gpio_write(handle->config.port_drvoff, handle->config.pin_drvoff, active);
00609
00610   return k_rx_ok;
00611 }
```

References [rx_drv8263_handle_t::config](#), [rx_drv8263_handle_t::initialized](#), [internal_gpio_write\(\)](#), [rx_drv8263_config_t::pin_drvoff](#), [rx_drv8263_config_t::port_drvoff](#), and [s_tag](#).

Here is the call graph for this function:



4.1.4.6 rx_drv8263_set_olp_boot_enable()

```
rx_err_t rx_drv8263_set_olp_boot_enable (
    rx_drv8263_handle_t * handle,
    bool enable) [nodiscard]
```

Enable or disable OLP diagnostic at boot time.

Toggles whether the Open Load Protection diagnostic runs during [rx_drv8263_init\(\)](#). When enabled, init will execute the full 3-pattern OLP sequence and log the result. This allows early detection of disconnected or shorted motors before the control loop begins.

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.
in	enable	true to enable boot OLP, false to disable

Returns

k_rx_ok on success

Return values

k_rx_err_null_ptr	if handle is nullptr
k_rx_err_invalid_state	if not initialized

Precondition

handle must not be nullptr

handle must be initialized via [rx_drv8263_init\(\)](#)

Postcondition

handle->config.olp_enable_boot updated to match enable parameter

No hardware side effects (configuration change only)

Note

Not thread-safe; caller must provide synchronization

Since

Version 1.0.0

Updates the olp_enable_boot flag in the driver configuration stored within the handle. When enabled, subsequent calls to [rx_drv8263_init\(\)](#) with this handle's configuration will execute the full 3-pattern OLP diagnostic sequence during initialization. This provides early detection of disconnected or shorted motors before the control loop begins.

This is a configuration-only change with no immediate hardware side effects. The flag takes effect the next time [rx_drv8263_init\(\)](#) is called with a configuration that has olp_enable_boot set.

Precondition

handle must be initialized via [rx_drv8263_init\(\)](#)
 handle must point to valid, writable memory

Postcondition

handle->config.olp_enable_boot == enable
 No hardware pins are modified (configuration change only)

Note

Not thread-safe; caller must provide synchronization when accessing the same handle from multiple contexts

See also

[rx_drv8263_init\(\)](#) Executes OLP at boot when olp_enable_boot is true
[rx_drv8263_set_olp_fault_enable\(\)](#) Companion function for fault-triggered OLP

Since

Version 1.0.0

Definition at line 819 of file [rx_drv8263.c](#).

```
00820 {
00821     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00822
00823     if (!handle->initialized) {
00824         rx_log_error(s_tag, "Not initialized");
00825         return k_rx_err_invalid_state;
00826     }
00827
00828     handle->config.olp_enable_boot = enable;
00829
00830     return k_rx_ok;
00831 }
```

References [rx_drv8263_handle_t::config](#), [rx_drv8263_handle_t::initialized](#), [rx_drv8263_config_t::olp_enable_boot](#), and [s_tag](#).

4.1.4.7 rx_drv8263_set_olp_fault_enable()

```
rx_err_t rx_drv8263_set_olp_fault_enable (
    rx\_drv8263\_handle\_t * handle,
    bool enable) [nodiscard]
```

Enable or disable OLP diagnostic in fault handler.

Toggles whether the Open Load Protection diagnostic runs after clearing a latched fault via [rx_drv8263_clear_latched_fault\(\)](#). When enabled, the fault handler will execute the 3-pattern OLP sequence to determine if the fault was caused by an open or shorted load condition.

Parameters

in	handle	Pointer to initialized handle. Must not be nullptr.
in	enable	true to enable fault OLP, false to disable

Returns

k_rx_ok on success

Return values

k_rx_err_null_ptr	if handle is nullptr
k_rx_err_invalid_state	if not initialized

Precondition

handle must not be nullptr

handle must be initialized via [rx_drv8263_init\(\)](#)

Postcondition

handle->config.olp_enable_fault updated to match enable parameter

No hardware side effects (configuration change only)

Note

Not thread-safe; caller must provide synchronization

Since

Version 1.0.0

Updates the olp_enable_fault flag in the driver configuration stored within the handle. When enabled, the fault handling path will execute the full 3-pattern OLP diagnostic sequence after clearing a latched fault via [rx_drv8263_clear_latched_fault\(\)](#). This helps determine whether the fault was caused by an open or shorted load condition.

This is a configuration-only change with no immediate hardware side effects. The flag is checked by the fault handling logic to decide whether to run OLP after fault clearance.

Precondition

handle must be initialized via [rx_drv8263_init\(\)](#)

handle must point to valid, writable memory

Postcondition

handle->config.olp_enable_fault == enable

No hardware pins are modified (configuration change only)

Note

Not thread-safe; caller must provide synchronization when accessing the same handle from multiple contexts

See also

[rx_drv8263_clear_latched_fault\(\)](#) Fault clearance function

[rx_drv8263_set_olp_boot_enable\(\)](#) Companion function for boot-time OLP

Since

Version 1.0.0

Definition at line 863 of file [rx_drv8263.c](#).

```
00864 {
00865     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00866
00867     if (!handle->initialized) {
00868         rx_log_error(s_tag, "Not initialized");
00869         return k_rx_err_invalid_state;
00870     }
00871
00872     handle->config.olp_enable_fault = enable;
00873
00874     return k_rx_ok;
00875 }
```

References [rx_drv8263_handle_t::config](#), [rx_drv8263_handle_t::initialized](#), [rx_drv8263_config_t::olp_enable_fault](#), and [s_tag](#).

4.2 rx_drv8263.h

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file rx_drv8263.h
00003  * @brief DRV8263H-Q1 Motor Driver Chip-Level Control API
00004  *
00005  * @details
00006  * ### Overview
00007  *
00008  * Provides chip-level control for the DRV8263H-Q1 H-bridge motor driver including:
00009  * - DRVOFF output disable control
00010  * - Latched fault reset via nSLEEP pulse
00011  * - Open Load Protection (OLP) diagnostics
00012  * - Current sensing ADC-to-ampere conversion
00013  *
00014  * This module complements rx_motor (which handles PWM/direction) by providing
00015  * DRV8263H-Q1-specific features not covered by the generic motor control API.
00016  *
00017  * ### Hardware Interface
00018  *
00019  * | Signal | Direction | Description |
00020  * |-----|-----|-----|
00021  * | DRVOFF | MCU -> DRV | Output disable (active-high, LOW = enabled) |
00022  * | nSLEEP | MCU -> DRV | Sleep control (active-low, HIGH = awake) |
00023  * | nFAULT | DRV -> MCU | Fault indicator (active-low, LOW = fault) |
00024  * | IN1 | MCU -> DRV | Half-bridge B control (PWM or GPIO for OLP) |
00025  * | IN2 | MCU -> DRV | Half-bridge A control (PWM or GPIO for OLP) |
00026  *
```

```

00027 * ## Current Sensing
00028 *
00029 * DRV8263H-Q1 IPROPI output characteristics:
00030 * - Current mirror ratio: 202 uA/A
00031 * - External sense resistor: 5100 ohm (5.1k)
00032 * -  $V\_IPROPI = I\_motor * 202e-6 * 5100 = I\_motor * 1.0302$ 
00033 * -  $I\_motor = V\_IPROPI / 1.0302$ 
00034 *
00035 * @par Module Dependencies:
00036 * - rx_err.h - Error code definitions
00037 * - rx_log.h - Structured logging
00038 * - rx_port_utils.h - GPIO register access
00039 *
00040 * @par NASA Power of 10 Compliance:
00041 * | Rule | Status | Details |
00042 * |-----|-----|-----|
00043 * | Rule 1 | [PASS] | No goto, setjmp, or recursion |
00044 * | Rule 2 | [PASS] | All loops bounded by enum constants |
00045 * | Rule 3 | [PASS] | Zero dynamic allocation |
00046 * | Rule 4 | [PASS] | All functions under 60 lines |
00047 * | Rule 5 | [PASS] | Minimum 2 validations per function |
00048 * | Rule 6 | [PASS] | Variables at smallest scope |
00049 * | Rule 7 | [PASS] | All return values checked |
00050 * | Rule 8 | [PASS] | C23 typed enums, minimal macros |
00051 * | Rule 9 | [WARN] | No function pointers in this module |
00052 * | Rule 10 | [PASS] | -Wall -Wextra -Werror clean |
00053 *
00054 * @see rx_motor.h Motor PWM control (complementary to this module)
00055 * @see hardware_config.h Pin assignments for DRVOFF, nSLEEP, nFAULT
00056 *
00057 * @author Locked, Inc.
00058 * @date 2026-03-03
00059 * @copyright Copyright (c) 2026 Locked Inc.
00060 * SPDX-License-Identifier: MIT
00061 * @version 1.0.0
00062 */
00063
00064 #pragma once
00065
00066 #include <stdbool.h>
00067 #include <stdint.h>
00068
00069 #include "rx_err.h"
00070
00071 #ifdef __cplusplus
00072 extern "C" {
00073 #endif
00074
00075 /*
=====
00076 * Constants
00077 *
=====
*/
00078
00079 /**
00080 * @enum rx_drv8263_ipropi_t
00081 * @brief DRV8263H-Q1 IPROPI current sensing constants
00082 *
00083 * @details
00084 * Constants for converting IPROPI analog voltage to motor current.
00085 *  $V\_IPROPI = I\_motor * k\_drv8263\_ipropi\_mirror\_ua\_per\_a * k\_drv8263\_ipropi\_sense\_ohm * 1e-6$ 
00086 *  $I\_motor = V\_IPROPI / 1.0302$ 
00087 *
00088 * @see rx_drv8263_adc_to_amps() Conversion helper function
00089 *
00090 * @code
00091 * float divisor = (float)k_drv8263_ipropi_mirror_ua_per_a
00092 *                 * (float)k_drv8263_ipropi_sense_ohm * 1e-6F;
00093 * // divisor == 1.0302F
00094 * @endcode
00095 *
00096 * @invariant k_drv8263_ipropi_mirror_ua_per_a == 202 (integer compile-time constant)
00097 * @invariant k_drv8263_ipropi_sense_ohm == 5100 (integer compile-time constant)
00098 *
00099 * @since Version 1.0.0
00100 */
00101 typedef enum : uint16_t {
00102     k_drv8263_ipropi_mirror_ua_per_a =
00103         202, /**< Current mirror ratio: 202 uA per amp of motor current */
00104     k_drv8263_ipropi_sense_ohm =
00105         5100, /**< IPROPI sense resistor value in ohms (5.1k, 1% tolerance) */
00106 } rx_drv8263_ipropi_t;
00107
00108 /**
00109 * @enum rx_drv8263_timing_us_t
00110 * @brief DRV8263H-Q1 timing constants in microseconds

```

```

00111 *
00112 * @details
00113 * Timing parameters from DRV8263H-Q1 datasheet for fault reset and OLP.
00114 *
00115 * @invariant k_drv8263_nsleep_pulse_us must be in range [5, 35] per datasheet
00116 *
00117 * @code
00118 * uint16_t pulse = k_drv8263_nsleep_pulse_us; // 20 us
00119 * uint16_t wake = k_drv8263_twake_us; // 1200 us
00120 * uint16_t settle = k_drv8263_olp_settle_us; // 50 us
00121 * @endcode
00122 *
00123 * @since Version 1.0.0
00124 */
00125 typedef enum : uint16_t {
00126     k_drv8263_nsleep_pulse_us =
00127         20, /**< nSLEEP LOW pulse duration for latched fault reset (target middle of 5-35 us window) */
00128     k_drv8263_twake_us =
00129         1200, /**< Wake-up time after nSLEEP HIGH (nFAULT returns HIGH, ~1.2 ms typ) */
00130     k_drv8263_olp_settle_us = 50, /**< Settling time between OLP test patterns (conservative) */
00131 } rx_drv8263_timing_us_t;
00132
00133 /**
00134 * @enum rx_drv8263_olp_pattern_count_t
00135 * @brief Number of OLP test patterns
00136 *
00137 * @details
00138 * The DRV8263H-Q1 OLP diagnostic uses exactly 3 test patterns (count=3).
00139 * Each pattern sets a unique IN1/IN2 combination while DRVOFF=HIGH, then
00140 * reads nFAULT to determine load condition:
00141 * - Pattern 0: IN1=0, IN2=0 (both inputs low, baseline reading)
00142 * - Pattern 1: IN1=1, IN2=0 (drives OUT1 side, detects open on OUT1)
00143 * - Pattern 2: IN1=0, IN2=1 (drives OUT2 side, detects open on OUT2)
00144 *
00145 * The 3-bit nFAULT result vector from these patterns is decoded against
00146 * the OLP truth table to classify the load as normal, open, shorted to
00147 * GND, or shorted to VM.
00148 *
00149 * @see rx_drv8263_run_olp() Executes the OLP diagnostic using these patterns
00150 * @see rx_drv8263_olp_result_t Result classification from pattern decoding
00151 *
00152 * @code
00153 * for (uint8_t i = 0; i < k_drv8263_olp_pattern_count; i++) {
00154 *     // apply pattern[i], read nFAULT
00155 * }
00156 * @endcode
00157 *
00158 * @invariant k_drv8263_olp_pattern_count == 3
00159 *
00160 * @since Version 1.0.0
00161 */
00162 typedef enum : uint8_t {
00163     k_drv8263_olp_pattern_count = 3, /**< Three test patterns for OLP diagnostic */
00164 } rx_drv8263_olp_pattern_count_t;
00165
00166 /**
=====
00167 * Types
00168 *
=====
*/
00169
00170 /**
00171 * @enum rx_drv8263_olp_result_t
00172 * @brief Open Load Protection diagnostic result per motor channel
00173 *
00174 * @details
00175 * Results from the DRV8263H-Q1 OLP diagnostic sequence. The 3-pattern test
00176 * with DRVOFF=HIGH reads nFAULT for each pattern to determine load condition.
00177 *
00178 * OLP Truth Table (nFAULT readings for patterns {IN1=0/IN2=0, IN1=1/IN2=0, IN1=0/IN2=1}):
00179 * - {1,1,1} = Normal (no fault)
00180 * - {1,1,0} = Open load on OUT2
00181 * - {1,0,1} = Open load on OUT1
00182 * - {0,1,1} = Short to GND
00183 * - {0,0,0} = Short to VM (or both shorted)
00184 *
00185 * @see rx_drv8263_run_olp() Executes OLP diagnostic
00186 *
00187 * @code
00188 * rx_drv8263_olp_result_t res = k_drv8263_olp_unknown;
00189 * // ... run OLP ...
00190 * if (res != k_drv8263_olp_normal) {
00191 *     // handle fault condition
00192 * }
00193 * @endcode
00194 *

```

```

00195 * @invariant Each value maps uniquely to a diagnostic state (0=normal, 1=open, 2=gnd, 3=vm, 4=unknown)
00196 *
00197 * @since Version 1.0.0
00198 */
00199 typedef enum : uint8_t {
00200     k_drv8263_olp_normal      = 0, /**< No fault detected, load connected normally */
00201     k_drv8263_olp_open_load   = 1, /**< Open load detected (motor disconnected) */
00202     k_drv8263_olp_short_to_gnd = 2, /**< Short circuit to ground detected */
00203     k_drv8263_olp_short_to_vm = 3, /**< Short circuit to motor supply (VM) detected */
00204     k_drv8263_olp_unknown     = 4, /**< Unknown or ambiguous diagnostic result */
00205 } rx_drv8263_olp_result_t;
00206
00207 /**
00208  * @struct rx_drv8263_config_t
00209  * @brief Configuration for a DRV8263H-Q1 motor driver instance
00210  *
00211  * @details
00212  * Specifies GPIO port/pin assignments for each control signal and OLP
00213  * diagnostic behavior flags. Port numbers use rx_port_utils.h constants
00214  * (e.g., k_rx_port_6 = 6, k_rx_port_e = 14).
00215  *
00216  * @par Usage Example:
00217  * @code
00218  * const rx_drv8263_config_t config = {
00219  *     .port_drvoft = 6, .pin_drvoft = 1, // P61
00220  *     .port_nsleep = 6, .pin_nsleep = 0, // P60
00221  *     .port_nfault = 1, .pin_nfault = 5, // P15
00222  *     .port_in1    = 1, .pin_in1    = 7, // P17
00223  *     .port_in2    = 2, .pin_in2    = 3, // P23
00224  *     .olp_enable_boot = true,
00225  *     .olp_enable_fault = true,
00226  * };
00227  * @endcode
00228  *
00229  * @invariant port_drvoft, port_nsleep, port_nfault, port_in1, port_in2 must be valid RX72N ports
00230  * @invariant pin values must be 0-7 (8-bit port width)
00231  *
00232  * @see rx_drv8263_init() Uses this configuration
00233  * @since Version 1.0.0
00234  */
00235 typedef struct {
00236     uint8_t port_drvoft; /**< DRVOFT GPIO port number (e.g., 6 for PORT6) */
00237     uint8_t pin_drvoft;  /**< DRVOFT GPIO pin number within port (0-7) */
00238     uint8_t port_nsleep; /**< nSLEEP GPIO port number */
00239     uint8_t pin_nsleep;  /**< nSLEEP GPIO pin number within port (0-7) */
00240     uint8_t port_nfault; /**< nFAULT GPIO port number (input, directly read for OLP) */
00241     uint8_t pin_nfault;  /**< nFAULT GPIO pin number within port (0-7) */
00242     uint8_t port_in1;    /**< IN1 GPIO port number (for OLP pattern output) */
00243     uint8_t pin_in1;     /**< IN1 GPIO pin number within port (0-7) */
00244     uint8_t port_in2;    /**< IN2 GPIO port number (for OLP pattern output) */
00245     uint8_t pin_in2;     /**< IN2 GPIO pin number within port (0-7) */
00246     bool   olp_enable_boot; /**< Run OLP diagnostic during rx_drv8263_init() */
00247     bool   olp_enable_fault; /**< Run OLP diagnostic after clearing latched faults */
00248 } rx_drv8263_config_t;
00249
00250 /**
00251  * @struct rx_drv8263_handle_t
00252  * @brief Runtime handle for a DRV8263H-Q1 motor driver instance
00253  *
00254  * @details
00255  * Stores configuration and initialization state. Allocated by caller
00256  * (typically static), initialized by rx_drv8263_init().
00257  *
00258  * @invariant initialized is false until rx_drv8263_init() succeeds
00259  *
00260  * @code
00261  * rx_drv8263_handle_t drv = {};
00262  * rx_err_t err = rx_drv8263_init(&drv, &config);
00263  * if (drv.initialized) { // ready to use }
00264  * @endcode
00265  *
00266  * @see rx_drv8263_init() Initializes handle
00267  * @since Version 1.0.0
00268  */
00269 typedef struct {
00270     rx_drv8263_config_t config; /**< Copied configuration (OLP flags mutable via setters) */
00271     bool               initialized; /**< true after successful init, false otherwise */
00272 } rx_drv8263_handle_t;
00273
00274 /**
00275  * Public API
00276  *
00277  *
00278  */

```

```

00279 * @brief Initialize DRV8263H-Q1 driver instance
00280 *
00281 * @details
00282 * Copies configuration, validates GPIO assignments, and optionally runs
00283 * OLP diagnostic if olp_enable_boot is set.
00284 *
00285 * @param[out] handle Pointer to handle structure. Must not be nullptr.
00286 * @param[in] config Pointer to configuration. Must not be nullptr.
00287 *
00288 * @return k_rx_ok on success
00289 * @retval k_rx_err_null_ptr if handle or config is nullptr
00290 * @retval k_rx_err_invalid_arg if GPIO port/pin values are invalid
00291 *
00292 * @pre handle must point to valid memory
00293 * @pre config must contain valid GPIO port/pin assignments
00294 *
00295 * @post handle->initialized = true on success
00296 * @post handle->config contains copy of *config
00297 *
00298 * @note Not thread-safe
00299 *
00300 * @since Version 1.0.0
00301 */
00302 [[nodiscard]] rx_err_t rx_drv8263_init(rx_drv8263_handle_t* handle,
00303                                     const rx_drv8263_config_t* config);
00304
00305 /**
00306 * @brief Control DRVOFF output disable signal
00307 *
00308 * @details
00309 * DRVOFF is active-high: setting active=true drives DRVOFF HIGH to disable
00310 * H-bridge outputs. Setting active=false drives DRVOFF LOW to enable outputs.
00311 *
00312 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00313 * @param[in] active true = DRVOFF HIGH (outputs disabled), false = DRVOFF LOW (outputs enabled)
00314 *
00315 * @return k_rx_ok on success
00316 * @retval k_rx_err_null_ptr if handle is nullptr
00317 * @retval k_rx_err_invalid_state if not initialized
00318 *
00319 * @pre handle must be initialized via rx_drv8263_init()
00320 * @pre DRVOFF GPIO must be configured as output (hardware_init responsibility)
00321 *
00322 * @post DRVOFF GPIO set to requested level
00323 * @post Handle remains initialized; no other H-bridge control pins modified
00324 *
00325 * @note Not thread-safe
00326 *
00327 * @since Version 1.0.0
00328 */
00329 [[nodiscard]] rx_err_t rx_drv8263_set_drvoff(rx_drv8263_handle_t* handle, bool active);
00330
00331 /**
00332 * @brief Clear latched fault by pulsing nSLEEP LOW
00333 *
00334 * @details
00335 * DRV8263H-Q1 latches faults until nSLEEP is pulsed LOW for 5-35 us.
00336 * This function:
00337 * 1. Drives nSLEEP LOW
00338 * 2. Busy-waits ~20 us (middle of 5-35 us window)
00339 * 3. Drives nSLEEP HIGH
00340 * 4. Sleeps one ThreadX tick (~10 ms with 100 Hz tick) to satisfy tWAKE (~1.2 ms);
00341 *    coarse-grained because tx_thread_sleep granularity is one tick
00342 *    (nFAULT is not polled -- it may be configured as a POEG input)
00343 *
00344 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00345 *
00346 * @return k_rx_ok on success
00347 * @retval k_rx_err_null_ptr if handle is nullptr
00348 * @retval k_rx_err_invalid_state if not initialized
00349 *
00350 * @pre handle must be initialized via rx_drv8263_init()
00351 * @pre Motor outputs should be stopped before clearing fault
00352 *
00353 * @post Latched fault cleared (if fault condition removed)
00354 * @post nSLEEP returns to HIGH (awake)
00355 *
00356 * @warning Uses CPU busy-wait for ~20 us pulse. Interrupts are NOT disabled
00357 *          during the pulse, so actual duration may be slightly longer.
00358 * @note Uses tx_thread_sleep(1) for tWAKE delay (10 ms tick >= 1.2 ms required)
00359 *
00360 * @since Version 1.0.0
00361 */
00362 [[nodiscard]] rx_err_t rx_drv8263_clear_latched_fault(rx_drv8263_handle_t* handle);
00363
00364 /**
00365 * @brief Run Open Load Protection diagnostic

```



```

00366 *
00367 * @details
00368 * Executes the DRV8263H-Q1 OLP diagnostic sequence with DRVOFF=HIGH:
00369 * 1. Set DRVOFF = HIGH (standby mode, enables OLP)
00370 * 2. For each of 3 test patterns, set IN1/IN2 and read nFAULT:
00371 *   - Pattern 0: IN1=0, IN2=0 -> read nFAULT
00372 *   - Pattern 1: IN1=1, IN2=0 -> read nFAULT
00373 *   - Pattern 2: IN1=0, IN2=1 -> read nFAULT
00374 * 3. Decode 3-bit nFAULT result against truth table
00375 * 4. Set DRVOFF = LOW (return to active mode)
00376 *
00377 * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00378 * @param[out] result_out1 OLP result for OUT1 (IN1 side). Must not be nullptr.
00379 * @param[out] result_out2 OLP result for OUT2 (IN2 side). Must not be nullptr.
00380 *
00381 * @return k_rx_ok on success
00382 * @retval k_rx_err_null_ptr if any pointer is nullptr
00383 * @retval k_rx_err_invalid_state if not initialized
00384 *
00385 * @pre handle must be initialized via rx_drv8263_init()
00386 * @pre Motor must be stopped (PWM disabled) before running OLP
00387 *
00388 * @post DRVOFF returns to LOW (outputs enabled)
00389 * @post result_out1 and result_out2 contain diagnostic results
00390 *
00391 * @warning IN1/IN2 pins are temporarily driven as GPIO during OLP.
00392 *         Motor PWM must be stopped before calling this function.
00393 * @note Execution time: ~200 us (3 patterns with settling delays)
00394 *
00395 * @code
00396 * rx_drv8263_handle_t drv;
00397 * // ... init drv ...
00398 *
00399 * // Stop PWM before running OLP
00400 * rx_drv8263_olp_result_t out1 = k_drv8263_olp_unknown;
00401 * rx_drv8263_olp_result_t out2 = k_drv8263_olp_unknown;
00402 * rx_err_t err = rx_drv8263_run_olp(&drv, &out1, &out2);
00403 * if (err == k_rx_ok) {
00404 *     if (out1 != k_drv8263_olp_normal) { // OUT1 fault
00405 *     }
00406 *     if (out2 != k_drv8263_olp_normal) { // OUT2 fault
00407 *     }
00408 * }
00409 * @endcode
00410 *
00411 * @since Version 1.0.0
00412 */
00413 [[nodiscard]] rx_err_t rx_drv8263_run_olp(rx_drv8263_handle_t* handle,
00414                                         rx_drv8263_olp_result_t* result_out1,
00415                                         rx_drv8263_olp_result_t* result_out2);
00416
00417 /**
00418 * @brief Convert IPROPI ADC voltage to motor current in amperes
00419 *
00420 * @details
00421 * Pure math conversion using DRV8263H-Q1 current sensing parameters:
00422 *  $I_{\text{motor}} = V_{\text{IPROPI}} / 1.0302$ 
00423 * where  $1.0302 = 202\text{e-}6 \text{ A/A} * 5100 \text{ ohm}$ 
00424 *
00425 * @param[in] adc_voltage_v IPROPI analog voltage in volts (0.0 to 3.3V)
00426 *
00427 * @return Motor current in amperes
00428 *
00429 * @pre adc_voltage_v must be non-negative
00430 * @pre adc_voltage_v should be within ADC range (0.0 to 3.3V typical)
00431 *
00432 * @post Return value  $\geq 0.0$ 
00433 * @post Return value is  $\text{adc\_voltage\_v} / k\_ipropi\_divisor$  (exact mathematical relationship)
00434 *
00435 * @note Pure function, thread-safe, no side effects
00436 *
00437 * @par Example:
00438 * @code
00439 * float voltage = adc_read_voltage(k_adc_ch_motor0_current);
00440 * float amps = rx_drv8263_adc_to_amps(voltage);
00441 * // 1.65V -> 1.602A
00442 * @endcode
00443 *
00444 * @since Version 1.0.0
00445 */
00446 static inline float rx_drv8263_adc_to_amps(float adc_voltage_v)
00447 {
00448     /* Combined IPROPI conversion factor:  $202\text{e-}6 * 5100 = 1.0302$  */
00449     static const float s_ipropi_divisor = 1.0302F;
00450
00451     return adc_voltage_v / s_ipropi_divisor;
00452 }

```

```

00453
00454 /**
00455  * @brief Enable or disable OLP diagnostic at boot time
00456  *
00457  * @details
00458  * Toggles whether the Open Load Protection diagnostic runs during
00459  * rx_drv8263_init(). When enabled, init will execute the full 3-pattern
00460  * OLP sequence and log the result. This allows early detection of
00461  * disconnected or shorted motors before the control loop begins.
00462  *
00463  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00464  * @param[in] enable true to enable boot OLP, false to disable
00465  *
00466  * @return k_rx_ok on success
00467  * @retval k_rx_err_null_ptr if handle is nullptr
00468  * @retval k_rx_err_invalid_state if not initialized
00469  *
00470  * @pre handle must not be nullptr
00471  * @pre handle must be initialized via rx_drv8263_init()
00472  *
00473  * @post handle->config.ulp_enable_boot updated to match enable parameter
00474  * @post No hardware side effects (configuration change only)
00475  *
00476  * @note Not thread-safe; caller must provide synchronization
00477  *
00478  * @since Version 1.0.0
00479  */
00480 [[nodiscard]] rx_err_t rx_drv8263_set_ulp_boot_enable(rx_drv8263_handle_t* handle, bool enable);
00481
00482 /**
00483  * @brief Enable or disable OLP diagnostic in fault handler
00484  *
00485  * @details
00486  * Toggles whether the Open Load Protection diagnostic runs after clearing
00487  * a latched fault via rx_drv8263_clear_latched_fault(). When enabled, the
00488  * fault handler will execute the 3-pattern OLP sequence to determine if
00489  * the fault was caused by an open or shorted load condition.
00490  *
00491  * @param[in] handle Pointer to initialized handle. Must not be nullptr.
00492  * @param[in] enable true to enable fault OLP, false to disable
00493  *
00494  * @return k_rx_ok on success
00495  * @retval k_rx_err_null_ptr if handle is nullptr
00496  * @retval k_rx_err_invalid_state if not initialized
00497  *
00498  * @pre handle must not be nullptr
00499  * @pre handle must be initialized via rx_drv8263_init()
00500  *
00501  * @post handle->config.ulp_enable_fault updated to match enable parameter
00502  * @post No hardware side effects (configuration change only)
00503  *
00504  * @note Not thread-safe; caller must provide synchronization
00505  *
00506  * @since Version 1.0.0
00507  */
00508 [[nodiscard]] rx_err_t rx_drv8263_set_ulp_fault_enable(rx_drv8263_handle_t* handle, bool enable);
00509
00510 /*
=====
00511  * Internal Functions (exposed for unit testing only)
00512  *
=====
*/
00513
00514 #ifdef UNIT_TEST
00515 #include "rx_check.h"
00516 void internal_delay_us(uint32_t us);
00517 rx_err_t internal_validate_config(const rx_drv8263_config_t* config);
00518 void internal_gpio_write(uint8_t port, uint8_t pin, bool high);
00519 bool internal_gpio_read(uint8_t port, uint8_t pin);
00520 void internal_olp_apply_patterns(const rx_drv8263_handle_t* handle, bool nfault_readings[]);
00521 void internal_olp_decode_results(bool f0,
00522                                  bool f1,
00523                                  bool f2,
00524                                  rx_drv8263_olp_result_t* result_out1,
00525                                  rx_drv8263_olp_result_t* result_out2);
00526 #endif /* UNIT_TEST */
00527
00528 #ifdef __cplusplus
00529 }
00530 #endif

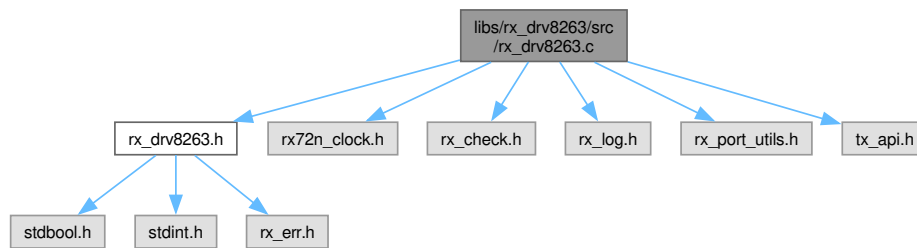
```

4.3 libs/rx_drv8263/src/rx_drv8263.c File Reference

DRV8263H-Q1 Motor Driver Chip-Level Control Implementation.

```
#include "rx_drv8263.h"
#include "rx72n_clock.h"
#include "rx_check.h"
#include "rx_log.h"
#include "rx_port_utils.h"
#include "tx_api.h"
```

Include dependency graph for rx_drv8263.c:



Enumerations

- enum `gpio_limits_t` : `uint8_t` { `k_max_port_number` = 16 , `k_max_pin_number` = 7 , `k_bit_shift_one` = 1 }
GPIO validation limits and bit-shift constants.
- enum `olp_nfault_idx_t` : `uint8_t` { `k_olp_pattern_both_low` = 0 , `k_olp_pattern_in1_high` = 1 , `k_olp_pattern_in2_high` = 2 }
Indices for OLP nFAULT reading array.
- enum `drv8263_delay_ticks_t` : `uint8_t` { `k_twake_threadx_ticks` }
ThreadX tick delay constants for DRV8263H-Q1 timing.

Functions

- RX_STATIC_TESTABLE void `internal_delay_us` (`uint32_t` us)
Busy-wait delay for microsecond-precision timing.
- static bool `internal_validate_gpio` (`uint8_t` port, `uint8_t` pin)
Validate GPIO port/pin pair.
- RX_STATIC_TESTABLE void `internal_gpio_write` (`uint8_t` port, `uint8_t` pin, bool high)
Set a GPIO output pin to HIGH or LOW.
- RX_STATIC_TESTABLE bool `internal_gpio_read` (`uint8_t` port, `uint8_t` pin)
Read a GPIO input pin state.
- RX_STATIC_TESTABLE `rx_err_t` `internal_validate_config` (const `rx_drv8263_config_t` *config)
Validate all GPIO assignments in config.
- RX_STATIC_TESTABLE void `internal_olp_apply_patterns` (const `rx_drv8263_handle_t` *handle, bool nfault_readings[])
Apply three OLP test patterns and read nFAULT for each.
- RX_STATIC_TESTABLE void `internal_olp_decode_results` (bool f0, bool f1, bool f2, `rx_drv8263_olp_result_t` *result_out1, `rx_drv8263_olp_result_t` *result_out2)
Decode OLP nFAULT readings into diagnostic results.

- `rx_err_t rx_drv8263_init (rx_drv8263_handle_t *handle, const rx_drv8263_config_t *config)`
Initialize DRV8263H-Q1 driver instance.
- `rx_err_t rx_drv8263_set_drvoff (rx_drv8263_handle_t *handle, bool active)`
Control DRVOFF output disable signal.
- `rx_err_t rx_drv8263_clear_latched_fault (rx_drv8263_handle_t *handle)`
Clear latched fault by pulsing nSLEEP LOW.
- `rx_err_t rx_drv8263_run_olp (rx_drv8263_handle_t *handle, rx_drv8263_olp_result_t *result_out1, rx_drv8263_olp_result_t *result_out2)`
Run Open Load Protection (OLP) diagnostic sequence.
- `rx_err_t rx_drv8263_set_olp_boot_enable (rx_drv8263_handle_t *handle, bool enable)`
Enable or disable OLP diagnostic at boot time.
- `rx_err_t rx_drv8263_set_olp_fault_enable (rx_drv8263_handle_t *handle, bool enable)`
Enable or disable OLP diagnostic in fault handler.

Variables

- static const char `s_tag []` = "DRV8263"
Logging tag for this module.

4.3.1 Detailed Description

DRV8263H-Q1 Motor Driver Chip-Level Control Implementation.

Implements DRVOFF control, latched fault reset via nSLEEP pulse, and Open Load Protection (OLP) diagnostics for the DRV8263H-Q1 H-bridge motor driver.

NASA Power of 10 Compliance:

Rule	Status	Details
Rule 1	↔ [PASS]↔	No goto, setjmp, or recursion
Rule 2	↔ [PASS]↔	All loops bounded by enum constants
Rule 3	↔ [PASS]↔	Zero dynamic allocation
Rule 4	↔ [PASS]↔	All functions under 60 lines
Rule 5	↔ [PASS]↔	Minimum 2 validations per function
Rule 6	↔ [PASS]↔	Variables at smallest scope

Rule	Status	Details
Rule 7	↔ [PASS]↔	All return values checked
Rule 8	↔ [PASS]↔	C23 typed enums, minimal macros
Rule 9	↔ [WARN]↔	No function pointers in this module
Rule 10	↔ [PASS]↔	-Wall -Wextra -Werror clean

SOLID Principles:

- S (Single Responsibility): Each helper handles one GPIO or decode task
- O (Open/Closed): Extensible via config struct, no hard-coded pins
- L (Liskov Substitution): N/A (no polymorphism in this module)
- I (Interface Segregation): Public API split into init, DRVOFF, fault, OLP
- D (Dependency Inversion): GPIO access via rx_port_utils abstraction layer

See also

[rx_drv8263.h](#) API documentation

Author

Locked, Inc.

Date

2026-03-03

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Version

1.0.0

Definition in file [rx_drv8263.c](#).

4.3.2 Enumeration Type Documentation

4.3.2.1 drv8263`delay`ticks`t

enum [drv8263_delay_ticks_t](#) : uint8_t

ThreadX tick delay constants for DRV8263H-Q1 timing.

The DRV8263H-Q1 requires a tWAKE period of approximately 1.2 ms after nSLEEP returns HIGH before nFAULT becomes valid. One ThreadX tick at 10 ms provides sufficient margin over the 1.2 ms specification.

See also

[rx_drv8263_clear_latched_fault\(\)](#) Uses [k_twake_threadx_ticks](#)

Since

Version 1.0.0

Enumerator

k_twake_threadx_ticks	ThreadX ticks to wait for tWAKE (~10 ms, margin over 1.2 ms spec)
---------------------------------------	---

Definition at line 135 of file [rx_drv8263.c](#).

```
00135     : uint8_t {
00136     k_twake_threadx_ticks =
00137     1, /**< ThreadX ticks to wait for tWAKE (~10 ms, margin over 1.2 ms spec) */
00138 } drv8263_delay_ticks_t;
```

4.3.2.2 gpio`limits`t

enum [gpio_limits_t](#) : uint8_t

GPIO validation limits and bit-shift constants.

Defines the maximum valid GPIO port and pin numbers for the RX72N microcontroller, as well as the unit bit-shift constant used for single-bit GPIO operations (set, clear, read).

```
if (port > k_max_port_number) { return false; }
base->podr |= (uint8_t)((uint8_t)k_bit_shift_one « pin);
```

See also

[internal_validate_gpio\(\)](#) Uses port/pin limits

[internal_gpio_write\(\)](#) Uses [k_bit_shift_one](#)

[internal_gpio_read\(\)](#) Uses [k_bit_shift_one](#)

Since

Version 1.0.0

Enumerator

k_max_port_number	Maximum valid port index (PORT0-PORTJ)
k_max_pin_number	Maximum valid pin within a port (0-7)
k_bit_shift_one	Unit bit value for single-bit GPIO mask operations

Definition at line 88 of file [rx_drv8263.c](#).

```
00088      : uint8_t {
00089  k_max_port_number = 16, /**< Maximum valid port index (PORT0-PORTJ) */
00090  k_max_pin_number = 7, /**< Maximum valid pin within a port (0-7) */
00091  k_bit_shift_one = 1, /**< Unit bit value for single-bit GPIO mask operations */
00092 } gpio_limits_t;
```

4.3.2.3 olp_nfault_idx_t

enum [olp_nfault_idx_t](#) : uint8_t

Indices for OLP nFAULT reading array.

Maps each OLP test pattern to its index in the `nfault_readings` array. The DRV8263H-Q1 OLP diagnostic uses three IN1/IN2 patterns while DRVOFF is HIGH; nFAULT is sampled after each pattern to determine the load condition on OUT1 and OUT2.

```
bool nfault_readings[k_drv8263_olp_pattern_count];
nfault_readings[k_olp_pattern_both_low] = read_nfault();
nfault_readings[k_olp_pattern_in1_high] = read_nfault();
nfault_readings[k_olp_pattern_in2_high] = read_nfault();
```

See also

[internal_olp_apply_patterns\(\)](#) Writes patterns and fills array

[internal_olp_decode_results\(\)](#) Decodes the three readings

Since

Version 1.0.0

Enumerator

k_olp_pattern_both_low	Pattern 0: IN1=0, IN2=0
k_olp_pattern_in1_high	Pattern 1: IN1=1, IN2=0
k_olp_pattern_in2_high	Pattern 2: IN1=0, IN2=1

Definition at line 116 of file [rx_drv8263.c](#).

```
00116      : uint8_t {
00117  k_olp_pattern_both_low = 0, /**< Pattern 0: IN1=0, IN2=0 */
00118  k_olp_pattern_in1_high = 1, /**< Pattern 1: IN1=1, IN2=0 */
00119  k_olp_pattern_in2_high = 2, /**< Pattern 2: IN1=0, IN2=1 */
00120 } olp_nfault_idx_t;
```

4.3.3 Function Documentation

4.3.3.1 internal_delay_us()

RX_STATIC_TESTABLE void internal_delay_us (
 uint32_t us)

Busy-wait delay for microsecond-precision timing.

CPU-cycle based busy-wait for short delays required by DRV8263H-Q1 timing specifications (nSLEEP pulse, OLP settling). At 240 MHz, each microsecond is approximately 240 CPU cycles.

Precondition

us must be strictly positive (us > 0; zero is not a valid delay)
us should be small (< 100 us) to avoid excessive blocking

Postcondition

Minimum us microseconds have elapsed
Timing tolerance: approximately 240 cycles per microsecond with possible additional cycles due to interrupts; interrupts remain in their original enable/disable state

Note

Interrupts are NOT disabled; actual delay may be slightly longer
Uses volatile counter to prevent compiler optimization

Since

Version 1.0.0

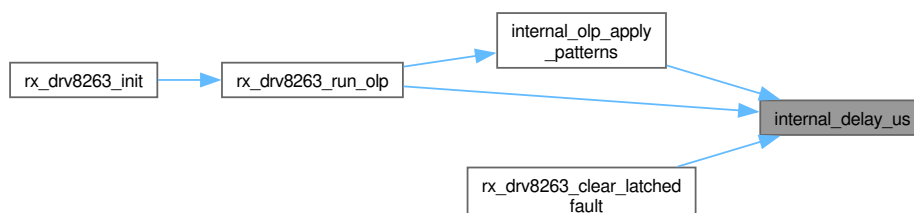
Maximum allowed busy-wait delay in microseconds

Definition at line 166 of file [rx_drv8263.c](#).

```
00167 {
00168     /** @brief Maximum allowed busy-wait delay in microseconds */
00169     enum : uint32_t {
00170         k_max_delay_us = 100,
00171         k_hz_per_mhz   = 1000000UL,
00172     };
00173
00174     if (us == 0 || us > k_max_delay_us) {
00175         return;
00176     }
00177
00178     /* Cycles per microsecond derive from the authoritative ICLK constant in
00179      * rx72n_clock.h so any future clock-tree change tracks automatically. */
00180     const uint32_t cycles_per_us = k_ickl_hz / k_hz_per_mhz;
00181     volatile uint32_t cycles      = us * cycles_per_us;
00182     while (cycles > 0) {
00183         cycles--;
00184     }
00185 }
```

Referenced by [internal_olp_apply_patterns\(\)](#), [rx_drv8263_clear_latched_fault\(\)](#), and [rx_drv8263_run_olp\(\)](#).

Here is the caller graph for this function:



4.3.3.2 `internal_gpio_read()`

```
RX_STATIC_TESTABLE bool internal_gpio_read (
    uint8_t port,
    uint8_t pin)
```

Read a GPIO input pin state.

Reads the port input data register (PIDR) and masks the target bit to determine pin state. Returns true if the pin is HIGH, false if the pin is LOW or the port base address is invalid (nullptr).

Precondition

port must be $\leq$ `k_max_port_number` and pin must be $\leq$ `k_max_pin_number`
 GPIO subsystem must be initialized (clock gating enabled for target port)

Postcondition

No side effects; return value reflects current pin state
 Port registers are not modified

Note

Thread safety: caller must provide synchronization if pin state can change asynchronously (e.g., from ISR context)

See also

[internal_gpio_write\(\)](#) Complementary write function

Since

Version 1.0.0

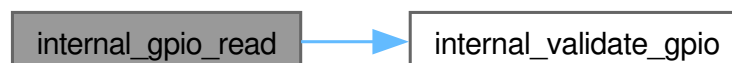
Definition at line 286 of file [rx_drv8263.c](#).

```
00287 {
00288     if (!internal_validate_gpio(port, pin)) {
00289         rx_log_debug(s_tag, "GPIO read: invalid port/pin");
00290         return false;
00291     }
00292     volatile rx_port_regs_t* base = rx_port_get_base(port);
00293     return (bool)((base->pidr & (uint8_t)(k_bit_shift_one « pin)) != 0);
00294 }
00295
00296 }
```

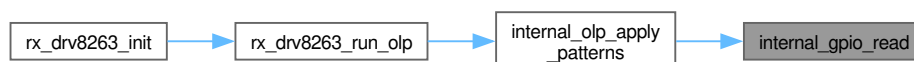
References [internal_validate_gpio\(\)](#), [k_bit_shift_one](#), and [s_tag](#).

Referenced by [internal_olp_apply_patterns\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.3 internal_gpio_write()

```
RX_STATIC_TESTABLE void internal_gpio_write (
    uint8_t port,
    uint8_t pin,
    bool high)
```

Set a GPIO output pin to HIGH or LOW.

Performs a read-modify-write on the port output data register (PODR) to set or clear a single pin without affecting other pins in the port. If the port base address is invalid (nullptr), the function returns silently with no effect.

Precondition

Port must be configured as output (PDR bit set)

port must be $\leq$ k_max_port_number and pin must be $\leq$ k_max_pin_number

Postcondition

Target pin state changed to requested level

No side-effects on other pins in the same port

Note

Not interrupt-safe; caller must provide synchronization if called from multiple contexts

See also

[internal_gpio_read\(\)](#) Complementary read function

Since

Version 1.0.0

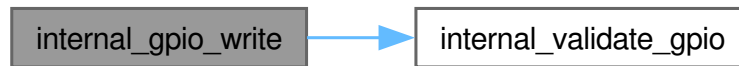
Definition at line 247 of file [rx_drv8263.c](#).

```
00248 {
00249     if (!internal_validate_gpio(port, pin)) {
00250         rx_log_debug(s_tag, "GPIO write: invalid port/pin");
00251         return;
00252     }
00253
00254     volatile rx_port_regs_t* base = rx_port_get_base(port);
00255
00256     if (high) {
00257         base->podr |= (uint8_t)(k_bit_shift_one « pin);
00258     } else {
00259         base->podr &= (uint8_t) ~(k_bit_shift_one « pin);
00260     }
00261 }
```

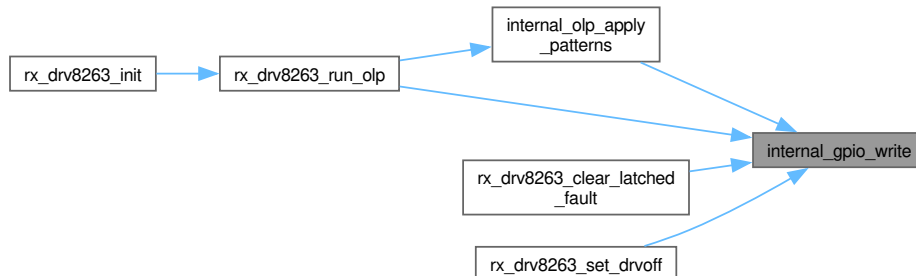
References [internal_validate_gpio\(\)](#), [k_bit_shift_one](#), and [s_tag](#).

Referenced by [internal_olp_apply_patterns\(\)](#), [rx_drv8263_clear_latched_fault\(\)](#), [rx_drv8263_run_olp\(\)](#), and [rx_drv8263_set_drvoff\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.4 internal_olp_apply_patterns()

```

RX_STATIC_TESTABLE void internal_olp_apply_patterns (
    const rx_drv8263_handle_t * handle,
    bool nfault_readings[])
  
```

Apply three OLP test patterns and read nFAULT for each.

Drives IN1 and IN2 through the three DRV8263H-Q1 OLP test patterns while DRVOFF is HIGH (caller is responsible for DRVOFF control). For each pattern, the function sets IN1/IN2, waits for the OLP settling time, and reads the nFAULT pin state into the corresponding element of the output array.

OLP test patterns:

- Pattern 0 (k_olp_pattern_both_low): IN1=0, IN2=0
- Pattern 1 (k_olp_pattern_in1_high): IN1=1, IN2=0
- Pattern 2 (k_olp_pattern_in2_high): IN1=0, IN2=1

Precondition

handle must be initialized via [rx_drv8263_init\(\)](#)

DRVOFF must already be set HIGH by caller (standby mode)

Postcondition

nfault_readings[0..2] populated with nFAULT readings

IN1 and IN2 are left in the state of the last pattern (IN1=0, IN2=1); caller must restore safe state

Note

Not thread-safe; caller must provide synchronization

See also

[rx_drv8263_run_olp\(\)](#) Main OLP function that calls this helper
[internal_olp_decode_results\(\)](#) Decodes the nFAULT readings

Since

Version 1.0.0

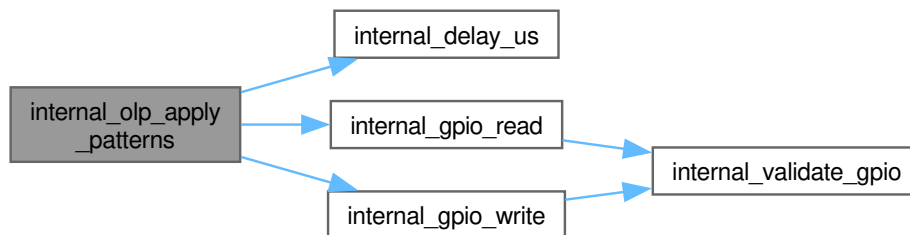
Definition at line 381 of file [rx_drv8263.c](#).

```
00383 {
00384     if (handle == nullptr || nfault_readings == nullptr) {
00385         return;
00386     }
00387
00388     /* Pattern 0: IN1=0, IN2=0 */
00389     internal_gpio_write(handle->config.port_in1, handle->config.pin_in1, false);
00390     internal_gpio_write(handle->config.port_in2, handle->config.pin_in2, false);
00391     internal_delay_us((uint32_t)k_drv8263_olp_settle_us);
00392     nfault_readings[k_olp_pattern_both_low] =
00393         internal_gpio_read(handle->config.port_nfault, handle->config.pin_nfault);
00394
00395     /* Pattern 1: IN1=1, IN2=0 */
00396     internal_gpio_write(handle->config.port_in1, handle->config.pin_in1, true);
00397     internal_gpio_write(handle->config.port_in2, handle->config.pin_in2, false);
00398     internal_delay_us((uint32_t)k_drv8263_olp_settle_us);
00399     nfault_readings[k_olp_pattern_in1_high] =
00400         internal_gpio_read(handle->config.port_nfault, handle->config.pin_nfault);
00401
00402     /* Pattern 2: IN1=0, IN2=1 */
00403     internal_gpio_write(handle->config.port_in1, handle->config.pin_in1, false);
00404     internal_gpio_write(handle->config.port_in2, handle->config.pin_in2, true);
00405     internal_delay_us((uint32_t)k_drv8263_olp_settle_us);
00406     nfault_readings[k_olp_pattern_in2_high] =
00407         internal_gpio_read(handle->config.port_nfault, handle->config.pin_nfault);
00408 }
```

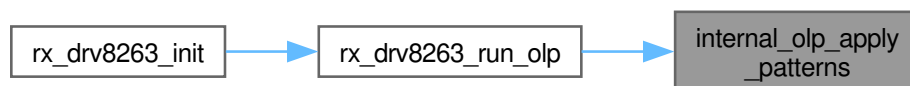
References [rx_drv8263_handle_t::config](#), [internal_delay_us\(\)](#), [internal_gpio_read\(\)](#), [internal_gpio_write\(\)](#), [k_drv8263_olp_settle_us](#), [k_olp_pattern_both_low](#), [k_olp_pattern_in1_high](#), [k_olp_pattern_in2_high](#), [rx_drv8263_config_t::pin_in1](#), [rx_drv8263_config_t::pin_in2](#), [rx_drv8263_config_t::pin_nfault](#), [rx_drv8263_config_t::port_in1](#), [rx_drv8263_config_t::port_in2](#), and [rx_drv8263_config_t::port_nfault](#).

Referenced by [rx_drv8263_run_olp\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.5 internal_olp_decode_results()

```
RX_STATIC_TESTABLE void internal_olp_decode_results (  
    bool f0,  
    bool f1,  
    bool f2,  
    rx_drv8263_olp_result_t * result_out1,  
    rx_drv8263_olp_result_t * result_out2)
```

Decode OLP nFAULT readings into diagnostic results.

Interprets the three nFAULT readings from the OLP test patterns using the DRV8263H-Q1 truth table to determine the load condition on OUT1 and OUT2. Logs warnings or errors for abnormal conditions.

Truth table (nFAULT readings for {pattern0, pattern1, pattern2}):

- {1,1,1} = Normal (no fault)
- {1,1,0} = Open Load OUT2
- {1,0,1} = Open Load OUT1
- {0,1,1} = Short to GND
- {0,0,0} = Short to VM (or both shorted)
- Other = Unknown / ambiguous

Precondition

result_out1 and result_out2 must point to valid memory
f0, f1, f2 must be valid nFAULT readings from OLP test patterns

Postcondition

result_out1 and result_out2 contain the decoded diagnostic results
A warning or error log is emitted for any abnormal condition

Note

Thread safety: no shared state; logging must be thread-safe

See also

[internal_olp_apply_patterns\(\)](#) Produces the nFAULT readings
[rx_drv8263_run_olp\(\)](#) Main OLP function that calls this helper

Since

Version 1.0.0

Definition at line 440 of file [rx_drv8263.c](#).

```

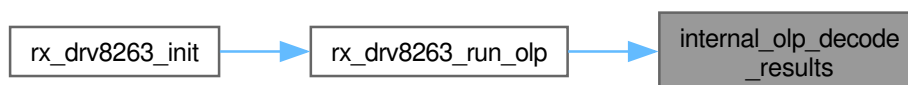
00445 {
00446     if (result_out1 == nullptr || result_out2 == nullptr) {
00447         rx_log_error(s_tag, "OLP: null output pointer");
00448         return;
00449     }
00450
00451     if ((int)f0 && (int)f1 && (int)f2) {
00452         /* {1,1,1} = Normal */
00453         *result_out1 = k_drv8263_olp_normal;
00454         *result_out2 = k_drv8263_olp_normal;
00455     } else if ((int)f0 && (int)f1) {
00456         /* {1,1,0} = Open Load on OUT2 (f2 must be false; f0&&f1&&f2 ruled out above) */
00457         *result_out1 = k_drv8263_olp_normal;
00458         *result_out2 = k_drv8263_olp_open_load;
00459         rx_log_warn(s_tag, "OLP: Open load on OUT2");
00460     } else if ((int)f0 && (int)f2) {
00461         /* {1,0,1} = Open Load on OUT1 (f1 must be false; f0&&f1 ruled out above) */
00462         *result_out1 = k_drv8263_olp_open_load;
00463         *result_out2 = k_drv8263_olp_normal;
00464         rx_log_warn(s_tag, "OLP: Open load on OUT1");
00465     } else if (!(int)f0 && (int)f1 && (int)f2) {
00466         /* {0,1,1} = Short to GND */
00467         *result_out1 = k_drv8263_olp_short_to_gnd;
00468         *result_out2 = k_drv8263_olp_short_to_gnd;
00469         rx_log_error(s_tag, "OLP: Short to GND detected");
00470     } else if (!(int)f0 && !(int)f1 && !(int)f2) {
00471         /* {0,0,0} = Short to VM */
00472         *result_out1 = k_drv8263_olp_short_to_vm;
00473         *result_out2 = k_drv8263_olp_short_to_vm;
00474         rx_log_error(s_tag, "OLP: Short to VM detected");
00475     } else {
00476         /* Unexpected pattern */
00477         *result_out1 = k_drv8263_olp_unknown;
00478         *result_out2 = k_drv8263_olp_unknown;
00479         rx_log_warn(s_tag, "OLP: Unknown nFAULT pattern");
00480     }
00481 }

```

References [k_drv8263_olp_normal](#), [k_drv8263_olp_open_load](#), [k_drv8263_olp_short_to_gnd](#), [k_drv8263_olp_short_to_vm](#), [k_drv8263_olp_unknown](#), and [s_tag](#).

Referenced by [rx_drv8263_run_olp\(\)](#).

Here is the caller graph for this function:



4.3.3.6 internal_validate_config()

```

RX_STATIC_TESTABLE rx_err_t internal_validate_config (
    const rx_drv8263_config_t * config)

```

Validate all GPIO assignments in config.

Iterates through all five GPIO assignments in the configuration struct (DRVOFF, nSLEEP, nFAULT, IN1, IN2) and validates each port/pin pair. Logs a descriptive error message identifying which signal has an invalid assignment on failure.

Precondition

config must not be nullptr (caller validates)
 config must point to a fully populated `rx_drv8263_config_t` struct

Postcondition

No modification to config (const pointer)
 On failure, exactly one error log message identifies the invalid signal

Note

Thread safety: pure validation, no shared state modified

See also

[internal_validate_gpio\(\)](#) Validates individual port/pin pairs

Since

Version 1.0.0

Definition at line 321 of file `rx_drv8263.c`.

```

00322 {
00323     if (config == nullptr) {
00324         return k_rx_err_null_ptr;
00325     }
00326
00327     if (internal_validate_gpio(config->port_drvoft, config->pin_drvoft)) {
00328         rx_log_error(s_tag, "Invalid DRVOFF GPIO");
00329         return k_rx_err_invalid_arg;
00330     }
00331     if (internal_validate_gpio(config->port_nsleap, config->pin_nsleap)) {
00332         rx_log_error(s_tag, "Invalid nSLEEP GPIO");
00333         return k_rx_err_invalid_arg;
00334     }
00335     if (internal_validate_gpio(config->port_nfault, config->pin_nfault)) {
00336         rx_log_error(s_tag, "Invalid nFAULT GPIO");
00337         return k_rx_err_invalid_arg;
00338     }
00339     if (internal_validate_gpio(config->port_in1, config->pin_in1)) {
00340         rx_log_error(s_tag, "Invalid IN1 GPIO");
00341         return k_rx_err_invalid_arg;
00342     }
00343     if (internal_validate_gpio(config->port_in2, config->pin_in2)) {
00344         rx_log_error(s_tag, "Invalid IN2 GPIO");
00345         return k_rx_err_invalid_arg;
00346     }
00347
00348     return k_rx_ok;
00349 }
```

References [internal_validate_gpio\(\)](#), [rx_drv8263_config_t::pin_drvoft](#), [rx_drv8263_config_t::pin_in1](#), [rx_drv8263_config_t::pin_in2](#), [rx_drv8263_config_t::pin_nfault](#), [rx_drv8263_config_t::pin_nsleap](#), [rx_drv8263_config_t::port_drvoft](#), [rx_drv8263_config_t::port_in1](#), [rx_drv8263_config_t::port_in2](#), [rx_drv8263_config_t::port_nfault](#), [rx_drv8263_config_t::port_nsleap](#), and [s_tag](#).

Referenced by [rx_drv8263_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.7 internal_validate_gpio()

```
bool internal_validate_gpio (
    uint8_t port,
    uint8_t pin)  [inline], [static]
```

Validate GPIO port/pin pair.

Checks that the given port and pin numbers fall within the valid range for the RX72N GPIO subsystem. Port must be 0-16 (PORT0 through PORTJ) and pin must be 0-7 (8-bit port width).

Precondition

port and pin are unsigned 8-bit values (always ≥ 0)
Called only during initialization or reconfiguration

Postcondition

No side effects; purely validates input range
Return value is deterministic for same inputs

Note

Thread safety: pure function, no shared state
This function intentionally does not use `assert()` for range checks because it IS the runtime validation function; callers rely on its return value to detect invalid GPIO coordinates.

See also

[internal_validate_config\(\)](#) Calls this for each GPIO assignment

Since

Version 1.0.0

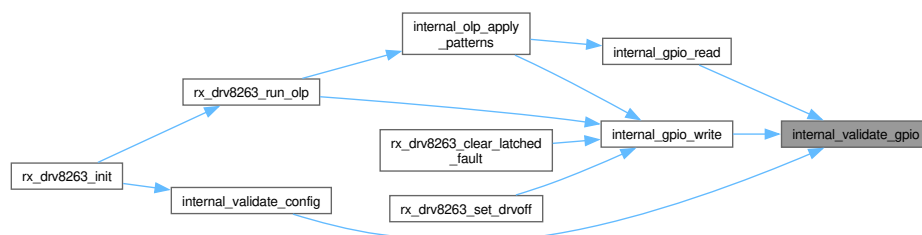
Definition at line 212 of file [rx_drv8263.c](#).

```
00213 {
00214     if (port > k_max_port_number) {
00215         return false;
00216     }
00217     if (pin > k_max_pin_number) {
00218         return false;
00219     }
00220     return true;
00221 }
```

References [k_max_pin_number](#), and [k_max_port_number](#).

Referenced by [internal_gpio_read\(\)](#), [internal_gpio_write\(\)](#), and [internal_validate_config\(\)](#).

Here is the caller graph for this function:



4.3.3.8 rx_drv8263_clear_latched_fault()

```
rx_err_t rx_drv8263_clear_latched_fault (  
    rx_drv8263_handle_t * handle) [nodiscard]
```

Clear latched fault by pulsing nSLEEP LOW.

The DRV8263H-Q1 latches faults (overcurrent, thermal shutdown) until the nSLEEP pin is pulsed LOW for a duration within the 5-35 us datasheet window. This function executes the following sequence:

1. Drive nSLEEP LOW to begin the fault reset pulse
2. Busy-wait for k_drv8263_nsleep_pulse_us (~20 us), targeting the middle of the 5-35 us specification window
3. Drive nSLEEP HIGH to wake the driver
4. Wait for tWAKE using tx_thread_sleep(k_twake_threadx_ticks). One ThreadX tick (~10 ms) provides sufficient margin over the ~1.2 ms tWAKE specification. After tWAKE, nFAULT returns HIGH if the underlying fault condition has been removed.

Timing Notes:

The nSLEEP pulse uses a CPU busy-wait loop (internal_delay_us) since the required precision (~20 us) is below ThreadX tick granularity (~10 ms). The tWAKE delay uses tx_thread_sleep to yield the CPU. nFAULT cannot be polled after the pulse because it may be configured as a POEG (Port Output Enable for GPT) input rather than a standard GPIO input.

Precondition

handle must be initialized via [rx_drv8263_init\(\)](#)

Motor outputs should be stopped (PWM disabled) before clearing a fault to prevent unexpected motor movement when the driver re-enables

Postcondition

nSLEEP pin is HIGH (driver awake)

Latched fault is cleared if the underlying fault condition has been removed

At least k_twake_threadx_ticks ThreadX ticks have elapsed (tWAKE satisfied)

Note

Not thread-safe; caller must provide synchronization

Cannot verify fault clearance because nFAULT may be a POEG input

See also

[rx_drv8263_init\(\)](#) Must be called before this function

[rx_drv8263_run_olp\(\)](#) Run OLP after clearing fault to diagnose load condition

[k_drv8263_nsleep_pulse_us](#) nSLEEP pulse duration constant

[k_twake_threadx_ticks](#) ThreadX tick delay for tWAKE

Since

Version 1.0.0

Definition at line 658 of file [rx_drv8263.c](#).

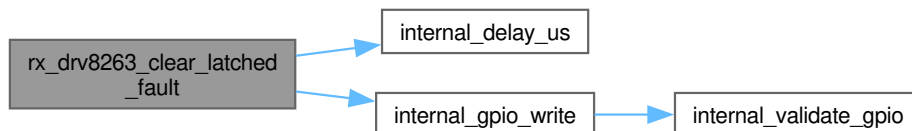
```

00659 {
00660     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00661
00662     if (!handle->initialized) {
00663         rx_log_error(s_tag, "Not initialized");
00664         return k_rx_err_invalid_state;
00665     }
00666
00667     /* Step 1: Drive nSLEEP LOW to begin fault reset pulse */
00668     internal_gpio_write(handle->config.port_nsleep, handle->config.pin_nsleep, false);
00669
00670     /* Step 2: Hold LOW for ~20 us (within 5-35 us spec window) */
00671     internal_delay_us((uint32_t)k_drv8263_nsleep_pulse_us);
00672
00673     /* Step 3: Drive nSLEEP HIGH to wake driver */
00674     internal_gpio_write(handle->config.port_nsleep, handle->config.pin_nsleep, true);
00675
00676     /* Step 4: Wait for tWAKE (~1.2 ms). One ThreadX tick = 10 ms provides
00677      * sufficient margin. Cannot poll nFAULT as it may be configured as POEG input. */
00678     /* NASA Rule 7: capture return value; intentionally discarded (sleep cannot
00679      * meaningfully fail in this context, but we document the discard). */
00680     const UINT sleep_status = tx_thread_sleep((ULONG)k_twake_threadx_ticks);
00681     (void)sleep_status;
00682
00683     rx_log_info(s_tag, "Latched fault cleared");
00684
00685     return k_rx_ok;
00686 }

```

References [rx_drv8263_handle_t::config](#), [rx_drv8263_handle_t::initialized](#), [internal_delay_us\(\)](#), [internal_gpio_write\(\)](#), [k_drv8263_nsleep_pulse_us](#), [k_twake_threadx_ticks](#), [rx_drv8263_config_t::pin_nsleep](#), [rx_drv8263_config_t::port_nsleep](#), and [s_tag](#).

Here is the call graph for this function:



4.3.3.9 rx_drv8263_init()

```

rx_err_t rx_drv8263_init (
    rx_drv8263_handle_t * handle,
    const rx_drv8263_config_t * config) [nodiscard]

```

Initialize DRV8263H-Q1 driver instance.

Validates all GPIO assignments in the provided configuration, copies the configuration into the handle, and marks the handle as initialized. If the configuration flag `olp_enable_boot` is set, the function additionally runs a full Open Load Protection (OLP) diagnostic sequence during initialization to detect disconnected or shorted motors before the control loop begins. OLP failures at boot are logged as warnings but do not prevent initialization from succeeding.

Initialization sequence:

1. Validate handle and config pointers (null check)
2. Validate all five GPIO port/pin pairs (DRVOFF, nSLEEP, nFAULT, IN1, IN2)
3. Copy configuration into handle
4. Set `handle->initialized = true`
5. (Optional) Run OLP diagnostic if `olp_enable_boot` is true

Precondition

handle must point to valid, writable memory of size $\geq \text{sizeof}(\text{rx_drv8263_handle_t})$
 config must contain valid GPIO port (0-16) and pin (0-7) assignments for all signals
 GPIO ports referenced in config must have their clocks enabled and pin directions set

Postcondition

handle->initialized == true on success
 handle->config contains a complete copy of *config
 On failure, handle is not modified (remains in its prior state)

Note

Not thread-safe; do not call concurrently for the same handle

Example:

```
static rx_drv8263_handle_t drv;
const rx_drv8263_config_t cfg = {
    .port_drvooff = 6, .pin_drvooff = 1,
    .port_nsleep = 6, .pin_nsleep = 0,
    .port_nfault = 1, .pin_nfault = 5,
    .port_in1 = 1, .pin_in1 = 7,
    .port_in2 = 2, .pin_in2 = 3,
    .olp_enable_boot = true,
    .olp_enable_fault = true,
};
rx_err_t err = rx_drv8263_init(&drv, &cfg);
```

See also

[rx_drv8263_set_drvooff\(\)](#) Control DRVOFF after initialization
[rx_drv8263_run_olp\(\)](#) Manual OLP diagnostic
[rx_drv8263_clear_latched_fault\(\)](#) Fault reset

Since

Version 1.0.0

Definition at line 539 of file [rx_drv8263.c](#).

```
00540 {
00541     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00542     RX_CHECK_NULL_PTR(config, s_tag, "config is nullptr");
00543
00544     const rx_err_t err = internal_validate_config(config);
00545     if (err != k_rx_ok) {
00546         return err;
00547     }
00548
00549     handle->config = *config;
00550     handle->initialized = true;
00551
00552     rx_log_info(s_tag, "DRV8263H-Q1 driver initialized");
00553
00554     /* Run OLP diagnostic at boot if configured */
00555     if (config->olp_enable_boot) {
00556         rx_drv8263_olp_result_t result_out1 = k_drv8263_olp_unknown;
00557         rx_drv8263_olp_result_t result_out2 = k_drv8263_olp_unknown;
00558
00559         (void)rx_drv8263_run_olp(handle, &result_out1, &result_out2);
00560         bool out1_abnormal = (bool)(result_out1 != k_drv8263_olp_normal);
```

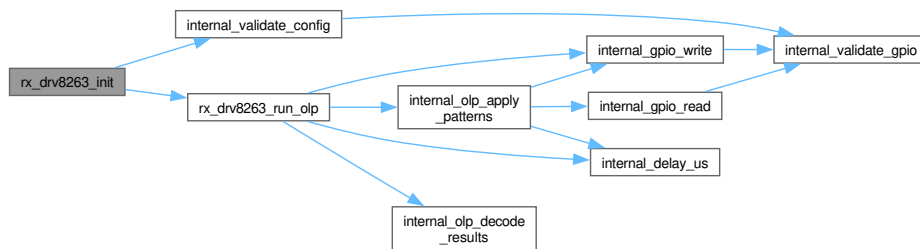
```

00561     bool out2_abnormal = (bool)(result_out2 != k_drv8263_olp_normal);
00562     if ((bool)((int)out1_abnormal | (int)out2_abnormal)) {
00563         rx_log_warn(s_tag, "Boot OLP detected abnormal load condition");
00564     }
00565 }
00566
00567 return k_rx_ok;
00568 }

```

References [rx_drv8263_handle_t::config](#), [rx_drv8263_handle_t::initialized](#), [internal_validate_config\(\)](#), [k_drv8263_olp_normal](#), [k_drv8263_olp_unknown](#), [rx_drv8263_config_t::olp_enable_boot](#), [rx_drv8263_run_olp\(\)](#), and [s_tag](#).

Here is the call graph for this function:



4.3.3.10 rx_drv8263_run_olp()

```

rx_err_t rx_drv8263_run_olp (
    rx_drv8263_handle_t * handle,
    rx_drv8263_olp_result_t * result_out1,
    rx_drv8263_olp_result_t * result_out2) [nodiscard]

```

Run Open Load Protection (OLP) diagnostic sequence.

Run Open Load Protection diagnostic.

Executes the full DRV8263H-Q1 OLP diagnostic to determine the load condition on motor outputs OUT1 and OUT2. The sequence temporarily places the driver in standby mode (DRVOFF HIGH), applies three IN1/IN2 test patterns while sampling nFAULT after each, then restores active mode (DRVOFF LOW).

Full OLP test sequence:

1. Set DRVOFF = HIGH (standby mode, enables OLP detection logic)
2. Wait k_drv8263_olp_settle_us (~50 us) for DRVOFF to take effect
3. Apply 3 test patterns, each followed by k_drv8263_olp_settle_us delay:
 - Pattern 0: IN1=0, IN2=0 -> read nFAULT (baseline)
 - Pattern 1: IN1=1, IN2=0 -> read nFAULT (tests OUT1 path)
 - Pattern 2: IN1=0, IN2=1 -> read nFAULT (tests OUT2 path)
4. Return IN1 and IN2 to LOW (safe state)
5. Set DRVOFF = LOW (return to active motor control mode)
6. Decode 3-bit nFAULT result vector against the truth table

nFAULT Pattern Ordering:

Patterns are applied in index order (`k_olp_pattern_both_low`, `k_olp_pattern_in1_high`, `k_olp_pattern_in2_high`). The nFAULT reading for each pattern is stored at the corresponding array index.

OLP Truth Table:

Pattern 0	Pattern 1	Pattern 2	OUT1 Result	OUT2 Result
1	1	1	Normal	Normal
1	1	0	Normal	Open Load
1	0	1	Open Load	Normal
0	1	1	Short to GND	Short to GND
0	0	0	Short to VM	Short to VM
other	other	other	Unknown	Unknown

Precondition

handle must be initialized via [rx_drv8263_init\(\)](#)

Motor PWM must be stopped (IN1/IN2 are driven as GPIO during OLP)

nFAULT pin must be readable as a standard GPIO input (not solely POEG)

Postcondition

DRVOFF returns to LOW (outputs re-enabled)

IN1 and IN2 are both LOW (safe / coast state)

result_out1 and result_out2 contain decoded OLP diagnostic results

Note

Not thread-safe; caller must provide synchronization

Execution time: approximately 200 us total (4 x `k_drv8263_olp_settle_us` settling delays plus GPIO read/write overhead)

See also

[rx_drv8263_init\(\)](#) Calls this at boot when `olp_enable_boot` is true

[rx_drv8263_set_drvoff\(\)](#) DRVOFF control used internally

[internal_olp_apply_patterns\(\)](#) Applies the 3 test patterns

[internal_olp_decode_results\(\)](#) Decodes the nFAULT readings

[k_drv8263_olp_settle_us](#) Settling delay between patterns

Since

Version 1.0.0

Definition at line 746 of file `rx_drv8263.c`.

```

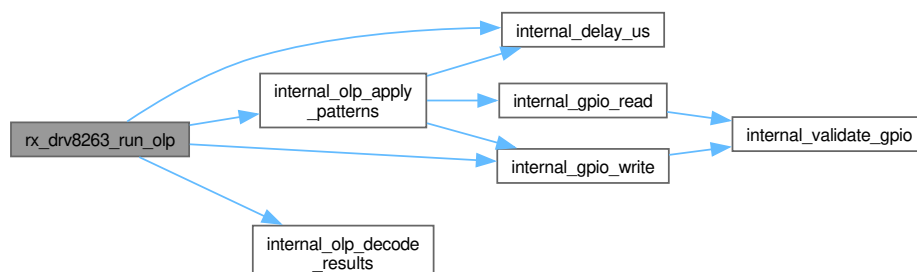
00749 {
00750     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00751     RX_CHECK_NULL_PTR(result_out1, s_tag, "result_out1 is nullptr");
00752     RX_CHECK_NULL_PTR(result_out2, s_tag, "result_out2 is nullptr");
00753
00754     if (!handle->initialized) {
00755         rx_log_error(s_tag, "Not initialized");
00756         return k_rx_err_invalid_state;
00757     }
00758
00759     /* Default results */
00760     *result_out1 = k_drv8263_olp_unknown;
00761     *result_out2 = k_drv8263_olp_unknown;
00762
00763     /* Step 1: Set DRVOFF = HIGH (standby mode, enables OLP detection) */
00764     internal_gpio_write(handle->config.port_drvoft, handle->config.pin_drvoft, true);
00765     internal_delay_us((uint32_t)k_drv8263_olp_settle_us);
00766
00767     /* Step 2: Apply 3 test patterns and read nFAULT for each */
00768     bool nfault_readings[k_drv8263_olp_pattern_count] = {false};
00769     internal_olp_apply_patterns(handle, nfault_readings);
00770
00771     /* Step 3: Return IN1/IN2 to LOW (safe state) */
00772     internal_gpio_write(handle->config.port_in1, handle->config.pin_in1, false);
00773     internal_gpio_write(handle->config.port_in2, handle->config.pin_in2, false);
00774
00775     /* Step 4: Set DRVOFF = LOW (return to active mode) */
00776     internal_gpio_write(handle->config.port_drvoft, handle->config.pin_drvoft, false);
00777
00778     /* Step 5: Decode OLP results from nFAULT readings */
00779     internal_olp_decode_results(nfault_readings[k_olp_pattern_both_low],
00780                                nfault_readings[k_olp_pattern_in1_high],
00781                                nfault_readings[k_olp_pattern_in2_high],
00782                                result_out1,
00783                                result_out2);
00784
00785     return k_rx_ok;
00786 }

```

References `rx_drv8263_handle_t::config`, `rx_drv8263_handle_t::initialized`, `internal_delay_us()`, `internal_gpio_write()`, `internal_olp_apply_patterns()`, `internal_olp_decode_results()`, `k_drv8263_olp_pattern_count`, `k_drv8263_olp_settle_us`, `k_drv8263_olp_unknown`, `k_olp_pattern_both_low`, `k_olp_pattern_in1_high`, `k_olp_pattern_in2_high`, `rx_drv8263_config_t::pin_drvoft`, `rx_drv8263_config_t::pin_in1`, `rx_drv8263_config_t::pin_in2`, `rx_drv8263_config_t::port_drvoft`, `rx_drv8263_config_t::port_in1`, `rx_drv8263_config_t::port_in2`, and `s_tag`.

Referenced by `rx_drv8263_init()`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.11 rx drv8263 set drvoff()

```
rx_err_t rx_drv8263_set_drvoff (
    rx_drv8263_handle_t * handle,
    bool active) [nodiscard]
```

Control DRVOFF output disable signal.

Drives the DRVOFF GPIO to enable or disable the DRV8263H-Q1 H-bridge outputs. DRVOFF is active-high: when active=true, the pin is driven HIGH and both half-bridge outputs (OUT1, OUT2) are placed in a high-impedance state. When active=false, the pin is driven LOW and normal PWM-controlled operation resumes.

DRVOFF must be asserted HIGH before running Open Load Protection (OLP) diagnostics, and de-asserted LOW to return to normal motor operation.

Precondition

handle must be initialized via [rx_drv8263_init\(\)](#)

DRVOFF GPIO must be configured as an output pin (hardware_init responsibility)

Postcondition

DRVOFF GPIO reflects the requested level (HIGH if active, LOW otherwise)

No other GPIO pins (nSLEEP, IN1, IN2) are modified

Note

Not thread-safe; caller must provide synchronization when accessing the same handle from multiple contexts

See also

[rx_drv8263_init\(\)](#) Must be called before this function

[rx_drv8263_run_olp\(\)](#) Internally uses DRVOFF HIGH for OLP sequence

Since

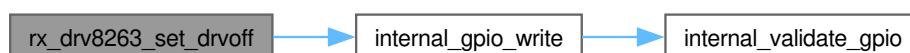
Version 1.0.0

Definition at line 599 of file [rx_drv8263.c](#).

```
00600 {
00601     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00602
00603     if (!handle->initialized) {
00604         rx_log_error(s_tag, "Not initialized");
00605         return k_rx_err_invalid_state;
00606     }
00607
00608     internal_gpio_write(handle->config.port_drvoff, handle->config.pin_drvoff, active);
00609
00610     return k_rx_ok;
00611 }
```

References [rx_drv8263_handle_t::config](#), [rx_drv8263_handle_t::initialized](#), [internal_gpio_write\(\)](#), [rx_drv8263_config_t::pin_drvoff](#), [rx_drv8263_config_t::port_drvoff](#), and [s_tag](#).

Here is the call graph for this function:



4.3.3.12 rx_drv8263_set_olp_boot_enable()

```
rx_err_t rx_drv8263_set_olp_boot_enable (
    rx_drv8263_handle_t * handle,
    bool enable) [nodiscard]
```

Enable or disable OLP diagnostic at boot time.

Updates the `olp_enable_boot` flag in the driver configuration stored within the handle. When enabled, subsequent calls to `rx_drv8263_init()` with this handle's configuration will execute the full 3-pattern OLP diagnostic sequence during initialization. This provides early detection of disconnected or shorted motors before the control loop begins.

This is a configuration-only change with no immediate hardware side effects. The flag takes effect the next time `rx_drv8263_init()` is called with a configuration that has `olp_enable_boot` set.

Precondition

handle must be initialized via `rx_drv8263_init()`
 handle must point to valid, writable memory

Postcondition

handle->config.olp_enable_boot == enable
 No hardware pins are modified (configuration change only)

Note

Not thread-safe; caller must provide synchronization when accessing the same handle from multiple contexts

See also

`rx_drv8263_init()` Executes OLP at boot when `olp_enable_boot` is true
`rx_drv8263_set_olp_fault_enable()` Companion function for fault-triggered OLP

Since

Version 1.0.0

Definition at line 819 of file `rx_drv8263.c`.

```
00820 {
00821     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00822
00823     if (!handle->initialized) {
00824         rx_log_error(s_tag, "Not initialized");
00825         return k_rx_err_invalid_state;
00826     }
00827
00828     handle->config.olp_enable_boot = enable;
00829
00830     return k_rx_ok;
00831 }
```

References `rx_drv8263_handle_t::config`, `rx_drv8263_handle_t::initialized`, `rx_drv8263_config_t::olp_enable_boot`, and `s_tag`.

4.3.3.13 rx_drv8263_set_olp_fault_enable()

```
rx_err_t rx_drv8263_set_olp_fault_enable (
    rx_drv8263_handle_t * handle,
    bool enable) [nodiscard]
```

Enable or disable OLP diagnostic in fault handler.

Updates the `olp_enable_fault` flag in the driver configuration stored within the handle. When enabled, the fault handling path will execute the full 3-pattern OLP diagnostic sequence after clearing a latched fault via [rx_drv8263_clear_latched_fault\(\)](#). This helps determine whether the fault was caused by an open or shorted load condition.

This is a configuration-only change with no immediate hardware side effects. The flag is checked by the fault handling logic to decide whether to run OLP after fault clearance.

Precondition

handle must be initialized via [rx_drv8263_init\(\)](#)
 handle must point to valid, writable memory

Postcondition

handle->config.olp_enable_fault == enable
 No hardware pins are modified (configuration change only)

Note

Not thread-safe; caller must provide synchronization when accessing the same handle from multiple contexts

See also

[rx_drv8263_clear_latched_fault\(\)](#) Fault clearance function
[rx_drv8263_set_olp_boot_enable\(\)](#) Companion function for boot-time OLP

Since

Version 1.0.0

Definition at line 863 of file [rx_drv8263.c](#).

```
00864 {
00865     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00866
00867     if (!handle->initialized) {
00868         rx_log_error(s_tag, "Not initialized");
00869         return k_rx_err_invalid_state;
00870     }
00871
00872     handle->config.olp_enable_fault = enable;
00873
00874     return k_rx_ok;
00875 }
```

References [rx_drv8263_handle_t::config](#), [rx_drv8263_handle_t::initialized](#), [rx_drv8263_config_t::olp_enable_fault](#), and [s_tag](#).

4.3.4 Variable Documentation

4.3.4.1 s_tag

```
const char s_tag[] = "DRV8263" [static]
```

Logging tag for this module.

Identifies DRV8263H-Q1 log messages in the structured logging output. Passed as the first argument to `rx_log_info()`, `rx_log_warn()`, `rx_log_error()`, and `rx_log_debug()` calls throughout this translation unit.

Note

Access restrictions: module-local (static), read-only after initialization

Warning

Must not be modified at runtime; changing this value would corrupt log output for the entire DRV8263 module

Since

Version 1.0.0

Definition at line 66 of file `rx_drv8263.c`.

Referenced by `internal_gpio_read()`, `internal_gpio_write()`, `internal_olp_decode_results()`, `internal_validate_config()`, `rx_drv8263_clear_latched_fault()`, `rx_drv8263_init()`, `rx_drv8263_run_olp()`, `rx_drv8263_set_drvoff()`, `rx_drv8263_set_olp_boot_enable()`, and `rx_drv8263_set_olp_fault_enable()`.

4.4 rx_drv8263.c

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file rx_drv8263.c
00003  * @brief DRV8263H-Q1 Motor Driver Chip-Level Control Implementation
00004  *
00005  * @details
00006  * Implements DRVOFF control, latched fault reset via nSLEEP pulse,
00007  * and Open Load Protection (OLP) diagnostics for the DRV8263H-Q1
00008  * H-bridge motor driver.
00009  *
00010  * @par NASA Power of 10 Compliance:
00011  * | Rule | Status | Details |
00012  * |-----|-----|-----|
00013  * | Rule 1 | [PASS] | No goto, setjmp, or recursion |
00014  * | Rule 2 | [PASS] | All loops bounded by enum constants |
00015  * | Rule 3 | [PASS] | Zero dynamic allocation |
00016  * | Rule 4 | [PASS] | All functions under 60 lines |
00017  * | Rule 5 | [PASS] | Minimum 2 validations per function |
00018  * | Rule 6 | [PASS] | Variables at smallest scope |
00019  * | Rule 7 | [PASS] | All return values checked |
00020  * | Rule 8 | [PASS] | C23 typed enums, minimal macros |
00021  * | Rule 9 | [WARN] | No function pointers in this module |
00022  * | Rule 10 | [PASS] | -Wall -Wextra -Werror clean |
00023  *
00024  * @par SOLID Principles:
00025  * - **S (Single Responsibility):** Each helper handles one GPIO or decode task
00026  * - **O (Open/Closed):** Extensible via config struct, no hard-coded pins
00027  * - **L (Liskov Substitution):** N/A (no polymorphism in this module)
00028  * - **I (Interface Segregation):** Public API split into init, DRVOFF, fault, OLP
```

```

00029 * - **D (Dependency Inversion):** GPIO access via rx_port_utils abstraction layer
00030 *
00031 * @see rx_drv8263.h API documentation
00032 *
00033 * @author Locked, Inc.
00034 * @date 2026-03-03
00035 * @copyright Copyright (c) 2026 Locked Inc.
00036 * SPDX-License-Identifier: MIT
00037 * @version 1.0.0
00038 */
00039
00040 #include "rx_drv8263.h"
00041
00042 #include "rx72n_clock.h"
00043 #include "rx_check.h"
00044 #include "rx_log.h"
00045 #ifdef UNIT_TEST
00046 #include "mock_rx_port_utils.h"
00047 #else
00048 #include "rx_port_utils.h"
00049 #endif
00050 #include "tx_api.h"
00051
00052 /**
00053 * @var s_tag
00054 * @brief Logging tag for this module
00055 *
00056 * @details
00057 * Identifies DRV8263H-Q1 log messages in the structured logging output.
00058 * Passed as the first argument to rx_log_info(), rx_log_warn(), rx_log_error(),
00059 * and rx_log_debug() calls throughout this translation unit.
00060 *
00061 * @note Access restrictions: module-local (static), read-only after initialization
00062 * @warning Must not be modified at runtime; changing this value would corrupt
00063 *         log output for the entire DRV8263 module
00064 * @since Version 1.0.0
00065 */
00066 static const char s_tag[] = "DRV8263";
00067
00068 /**
00069 * @enum gpio_limits_t
00070 * @brief GPIO validation limits and bit-shift constants
00071 *
00072 * @details
00073 * Defines the maximum valid GPIO port and pin numbers for the RX72N
00074 * microcontroller, as well as the unit bit-shift constant used for
00075 * single-bit GPIO operations (set, clear, read).
00076 *
00077 * @code
00078 * if (port > k_max_port_number) { return false; }
00079 * base->podr |= (uint8_t)((uint8_t)k_bit_shift_one « pin);
00080 * @endcode
00081 *
00082 * @see internal_validate_gpio() Uses port/pin limits
00083 * @see internal_gpio_write() Uses k_bit_shift_one
00084 * @see internal_gpio_read() Uses k_bit_shift_one
00085 *
00086 * @since Version 1.0.0
00087 */
00088 typedef enum : uint8_t {
00089     k_max_port_number = 16, /**< Maximum valid port index (PORT0-PORTJ) */
00090     k_max_pin_number = 7, /**< Maximum valid pin within a port (0-7) */
00091     k_bit_shift_one = 1, /**< Unit bit value for single-bit GPIO mask operations */
00092 } gpio_limits_t;
00093
00094 /**
00095 * @enum olp_nfault_idx_t
00096 * @brief Indices for OLP nFAULT reading array
00097 *
00098 * @details
00099 * Maps each OLP test pattern to its index in the nfault_readings array.
00100 * The DRV8263H-Q1 OLP diagnostic uses three IN1/IN2 patterns while
00101 * DRVOFF is HIGH; nFAULT is sampled after each pattern to determine
00102 * the load condition on OUT1 and OUT2.
00103 *
00104 * @code
00105 * bool nfault_readings[k_drv8263_olp_pattern_count];
00106 * nfault_readings[k_olp_pattern_both_low] = read_nfault();
00107 * nfault_readings[k_olp_pattern_in1_high] = read_nfault();
00108 * nfault_readings[k_olp_pattern_in2_high] = read_nfault();
00109 * @endcode
00110 *
00111 * @see internal_olp_apply_patterns() Writes patterns and fills array
00112 * @see internal_olp_decode_results() Decodes the three readings
00113 *
00114 * @since Version 1.0.0
00115 */

```

```

00116 typedef enum : uint8_t {
00117     k_olp_pattern_both_low = 0, /**< Pattern 0: IN1=0, IN2=0 */
00118     k_olp_pattern_in1_high = 1, /**< Pattern 1: IN1=1, IN2=0 */
00119     k_olp_pattern_in2_high = 2, /**< Pattern 2: IN1=0, IN2=1 */
00120 } olp_nfault_idx_t;
00121
00122 /**
00123  * @enum drv8263_delay_ticks_t
00124  * @brief ThreadX tick delay constants for DRV8263H-Q1 timing
00125  *
00126  * @details
00127  * The DRV8263H-Q1 requires a tWAKE period of approximately 1.2 ms after
00128  * nSLEEP returns HIGH before nFAULT becomes valid. One ThreadX tick at
00129  * 10 ms provides sufficient margin over the 1.2 ms specification.
00130  *
00131  * @see rx_drv8263_clear_latched_fault() Uses k_twake_threadx_ticks
00132  *
00133  * @since Version 1.0.0
00134  */
00135 typedef enum : uint8_t {
00136     k_twake_threadx_ticks =
00137         1, /**< ThreadX ticks to wait for tWAKE (~10 ms, margin over 1.2 ms spec) */
00138 } drv8263_delay_ticks_t;
00139
00140 /*
00141  * Internal Helper Functions
00142  */
00143
00144 /**
00145  * @brief Busy-wait delay for microsecond-precision timing
00146  *
00147  * @details
00148  * CPU-cycle based busy-wait for short delays required by DRV8263H-Q1
00149  * timing specifications (nSLEEP pulse, OLP settling). At 240 MHz,
00150  * each microsecond is approximately 240 CPU cycles.
00151  *
00152  * @pre us must be strictly positive (us > 0; zero is not a valid delay)
00153  * @pre us should be small (< 100 us) to avoid excessive blocking
00154  *
00155  * @post Minimum us microseconds have elapsed
00156  * @post Timing tolerance: approximately 240 cycles per microsecond with
00157  *       possible additional cycles due to interrupts; interrupts remain
00158  *       in their original enable/disable state
00159  *
00160  * @note Interrupts are NOT disabled; actual delay may be slightly longer
00161  * @note Uses volatile counter to prevent compiler optimization
00162  *
00163  * @since Version 1.0.0
00164  */
00165
00166 RX_STATIC_TESTABLE void internal_delay_us(uint32_t us)
00167 {
00168     /** @brief Maximum allowed busy-wait delay in microseconds */
00169     enum : uint32_t {
00170         k_max_delay_us = 100,
00171         k_hz_per_mhz   = 1000000UL,
00172     };
00173
00174     if (us == 0 || us > k_max_delay_us) {
00175         return;
00176     }
00177
00178     /* Cycles per microsecond derive from the authoritative ICLK constant in
00179      * rx72n_clock.h so any future clock-tree change tracks automatically. */
00180     const uint32_t cycles_per_us = k_iclk_hz / k_hz_per_mhz;
00181     volatile uint32_t cycles      = us * cycles_per_us;
00182     while (cycles > 0) {
00183         cycles--;
00184     }
00185 }
00186
00187 /**
00188  * @brief Validate GPIO port/pin pair
00189  *
00190  * @details
00191  * Checks that the given port and pin numbers fall within the valid range
00192  * for the RX72N GPIO subsystem. Port must be 0-16 (PORT0 through PORTJ)
00193  * and pin must be 0-7 (8-bit port width).
00194  *
00195  * @pre port and pin are unsigned 8-bit values (always >= 0)
00196  * @pre Called only during initialization or reconfiguration
00197  */
00198
00199

```

```

00200 * @post No side effects; purely validates input range
00201 * @post Return value is deterministic for same inputs
00202 *
00203 * @note Thread safety: pure function, no shared state
00204 * @note This function intentionally does not use assert() for range checks
00205 *         because it IS the runtime validation function; callers rely on its
00206 *         return value to detect invalid GPIO coordinates.
00207 *
00208 * @see internal_validate_config() Calls this for each GPIO assignment
00209 *
00210 * @since Version 1.0.0
00211 */
00212 static inline bool internal_validate_gpio(uint8_t port, uint8_t pin)
00213 {
00214     if (port > k_max_port_number) {
00215         return false;
00216     }
00217     if (pin > k_max_pin_number) {
00218         return false;
00219     }
00220     return true;
00221 }
00222
00223 /**
00224 * @brief Set a GPIO output pin to HIGH or LOW
00225 *
00226 * @details
00227 * Performs a read-modify-write on the port output data register (PODR)
00228 * to set or clear a single pin without affecting other pins in the port.
00229 * If the port base address is invalid (nullptr), the function returns
00230 * silently with no effect.
00231 *
00232 *
00233 *
00234 * @pre Port must be configured as output (PDR bit set)
00235 * @pre port must be <= k_max_port_number and pin must be <= k_max_pin_number
00236 *
00237 * @post Target pin state changed to requested level
00238 * @post No side-effects on other pins in the same port
00239 *
00240 * @note Not interrupt-safe; caller must provide synchronization if called
00241 *         from multiple contexts
00242 *
00243 * @see internal_gpio_read() Complementary read function
00244 *
00245 * @since Version 1.0.0
00246 */
00247 RX_STATIC_TESTABLE void internal_gpio_write(uint8_t port, uint8_t pin, bool high)
00248 {
00249     if (!internal_validate_gpio(port, pin)) {
00250         rx_log_debug(s_tag, "GPIO write: invalid port/pin");
00251         return;
00252     }
00253
00254     volatile rx_port_regs_t* base = rx_port_get_base(port);
00255
00256     if (high) {
00257         base->podr |= (uint8_t)(k_bit_shift_one << pin);
00258     } else {
00259         base->podr &= (uint8_t) ~(k_bit_shift_one << pin);
00260     }
00261 }
00262
00263 /**
00264 * @brief Read a GPIO input pin state
00265 *
00266 * @details
00267 * Reads the port input data register (PIDR) and masks the target bit to
00268 * determine pin state. Returns true if the pin is HIGH, false if the pin
00269 * is LOW or the port base address is invalid (nullptr).
00270 *
00271 *
00272 *
00273 * @pre port must be <= k_max_port_number and pin must be <= k_max_pin_number
00274 * @pre GPIO subsystem must be initialized (clock gating enabled for target port)
00275 *
00276 * @post No side effects; return value reflects current pin state
00277 * @post Port registers are not modified
00278 *
00279 * @note Thread safety: caller must provide synchronization if pin state
00280 *         can change asynchronously (e.g., from ISR context)
00281 *
00282 * @see internal_gpio_write() Complementary write function
00283 *
00284 * @since Version 1.0.0
00285 */
00286 RX_STATIC_TESTABLE bool internal_gpio_read(uint8_t port, uint8_t pin)

```

```

00287 {
00288     if (!internal_validate_gpio(port, pin)) {
00289         rx_log_debug(s_tag, "GPIO read: invalid port/pin");
00290         return false;
00291     }
00292
00293     volatile rx_port_regs_t* base = rx_port_get_base(port);
00294
00295     return (bool)((base->pidr & (uint8_t)(k_bit_shift_one « pin)) != 0);
00296 }
00297
00298 /**
00299  * @brief Validate all GPIO assignments in config
00300  *
00301  * @details
00302  * Iterates through all five GPIO assignments in the configuration struct
00303  * (DRVOFF, nSLEEP, nFAULT, IN1, IN2) and validates each port/pin pair.
00304  * Logs a descriptive error message identifying which signal has an invalid
00305  * assignment on failure.
00306  *
00307  *
00308  *
00309  * @pre config must not be nullptr (caller validates)
00310  * @pre config must point to a fully populated rx_drv8263_config_t struct
00311  *
00312  * @post No modification to config (const pointer)
00313  * @post On failure, exactly one error log message identifies the invalid signal
00314  *
00315  * @note Thread safety: pure validation, no shared state modified
00316  *
00317  * @see internal_validate_gpio() Validates individual port/pin pairs
00318  *
00319  * @since Version 1.0.0
00320  */
00321 RX_STATIC_TESTABLE rx_err_t internal_validate_config(const rx_drv8263_config_t* config)
00322 {
00323     if (config == nullptr) {
00324         return k_rx_err_null_ptr;
00325     }
00326
00327     if (!internal_validate_gpio(config->port_drvoft, config->pin_drvoft)) {
00328         rx_log_error(s_tag, "Invalid DRVOFF GPIO");
00329         return k_rx_err_invalid_arg;
00330     }
00331     if (!internal_validate_gpio(config->port_nsleee, config->pin_nsleee)) {
00332         rx_log_error(s_tag, "Invalid nSLEEP GPIO");
00333         return k_rx_err_invalid_arg;
00334     }
00335     if (!internal_validate_gpio(config->port_nfault, config->pin_nfault)) {
00336         rx_log_error(s_tag, "Invalid nFAULT GPIO");
00337         return k_rx_err_invalid_arg;
00338     }
00339     if (!internal_validate_gpio(config->port_in1, config->pin_in1)) {
00340         rx_log_error(s_tag, "Invalid IN1 GPIO");
00341         return k_rx_err_invalid_arg;
00342     }
00343     if (!internal_validate_gpio(config->port_in2, config->pin_in2)) {
00344         rx_log_error(s_tag, "Invalid IN2 GPIO");
00345         return k_rx_err_invalid_arg;
00346     }
00347
00348     return k_rx_ok;
00349 }
00350
00351 /**
00352  * @brief Apply three OLP test patterns and read nFAULT for each
00353  *
00354  * @details
00355  * Drives IN1 and IN2 through the three DRV8263H-Q1 OLP test patterns
00356  * while DRVOFF is HIGH (caller is responsible for DRVOFF control).
00357  * For each pattern, the function sets IN1/IN2, waits for the OLP
00358  * settling time, and reads the nFAULT pin state into the corresponding
00359  * element of the output array.
00360  *
00361  * OLP test patterns:
00362  * - Pattern 0 (k_olp_pattern_both_low): IN1=0, IN2=0
00363  * - Pattern 1 (k_olp_pattern_in1_high): IN1=1, IN2=0
00364  * - Pattern 2 (k_olp_pattern_in2_high): IN1=0, IN2=1
00365  *
00366  *
00367  * @pre handle must be initialized via rx_drv8263_init()
00368  * @pre DRVOFF must already be set HIGH by caller (standby mode)
00369  *
00370  * @post nfault_readings[0..2] populated with nFAULT readings
00371  * @post IN1 and IN2 are left in the state of the last pattern (IN1=0, IN2=1);
00372  *       caller must restore safe state
00373  *

```

```

00374 * @note Not thread-safe; caller must provide synchronization
00375 *
00376 * @see rx_drv8263_run_olp() Main OLP function that calls this helper
00377 * @see internal_olp_decode_results() Decodes the nFAULT readings
00378 *
00379 * @since Version 1.0.0
00380 */
00381 RX_STATIC_TESTABLE void internal_olp_apply_patterns(const rx_drv8263_handle_t* handle,
00382                                                    bool nfault_readings[])
00383 {
00384     if (handle == nullptr || nfault_readings == nullptr) {
00385         return;
00386     }
00387
00388     /* Pattern 0: IN1=0, IN2=0 */
00389     internal_gpio_write(handle->config.port_in1, handle->config.pin_in1, false);
00390     internal_gpio_write(handle->config.port_in2, handle->config.pin_in2, false);
00391     internal_delay_us((uint32_t)k_drv8263_olp_settle_us);
00392     nfault_readings[k_olp_pattern_both_low] =
00393         internal_gpio_read(handle->config.port_nfault, handle->config.pin_nfault);
00394
00395     /* Pattern 1: IN1=1, IN2=0 */
00396     internal_gpio_write(handle->config.port_in1, handle->config.pin_in1, true);
00397     internal_gpio_write(handle->config.port_in2, handle->config.pin_in2, false);
00398     internal_delay_us((uint32_t)k_drv8263_olp_settle_us);
00399     nfault_readings[k_olp_pattern_in1_high] =
00400         internal_gpio_read(handle->config.port_nfault, handle->config.pin_nfault);
00401
00402     /* Pattern 2: IN1=0, IN2=1 */
00403     internal_gpio_write(handle->config.port_in1, handle->config.pin_in1, false);
00404     internal_gpio_write(handle->config.port_in2, handle->config.pin_in2, true);
00405     internal_delay_us((uint32_t)k_drv8263_olp_settle_us);
00406     nfault_readings[k_olp_pattern_in2_high] =
00407         internal_gpio_read(handle->config.port_nfault, handle->config.pin_nfault);
00408 }
00409
00410 /**
00411 * @brief Decode OLP nFAULT readings into diagnostic results
00412 *
00413 * @details
00414 * Interprets the three nFAULT readings from the OLP test patterns using
00415 * the DRV8263H-Q1 truth table to determine the load condition on OUT1
00416 * and OUT2. Logs warnings or errors for abnormal conditions.
00417 *
00418 * Truth table (nFAULT readings for {pattern0, pattern1, pattern2}):
00419 * - {1,1,1} = Normal (no fault)
00420 * - {1,1,0} = Open Load OUT2
00421 * - {1,0,1} = Open Load OUT1
00422 * - {0,1,1} = Short to GND
00423 * - {0,0,0} = Short to VM (or both shorted)
00424 * - Other = Unknown / ambiguous
00425 *
00426 *
00427 * @pre result_out1 and result_out2 must point to valid memory
00428 * @pre f0, f1, f2 must be valid nFAULT readings from OLP test patterns
00429 *
00430 * @post result_out1 and result_out2 contain the decoded diagnostic results
00431 * @post A warning or error log is emitted for any abnormal condition
00432 *
00433 * @note Thread safety: no shared state; logging must be thread-safe
00434 *
00435 * @see internal_olp_apply_patterns() Produces the nFAULT readings
00436 * @see rx_drv8263_run_olp() Main OLP function that calls this helper
00437 *
00438 * @since Version 1.0.0
00439 */
00440 RX_STATIC_TESTABLE void internal_olp_decode_results(bool f0,
00441                                                    bool f1,
00442                                                    bool f2,
00443                                                    rx_drv8263_olp_result_t* result_out1,
00444                                                    rx_drv8263_olp_result_t* result_out2)
00445 {
00446     if (result_out1 == nullptr || result_out2 == nullptr) {
00447         rx_log_error(s_tag, "OLP: null output pointer");
00448         return;
00449     }
00450
00451     if ((int)f0 && (int)f1 && (int)f2) {
00452         /* {1,1,1} = Normal */
00453         *result_out1 = k_drv8263_olp_normal;
00454         *result_out2 = k_drv8263_olp_normal;
00455     } else if ((int)f0 && (int)f1) {
00456         /* {1,1,0} = Open Load on OUT2 (f2 must be false; f0&&f1&&f2 ruled out above) */
00457         *result_out1 = k_drv8263_olp_normal;
00458         *result_out2 = k_drv8263_olp_open_load;
00459         rx_log_warn(s_tag, "OLP: Open load on OUT2");
00460     } else if ((int)f0 && (int)f2) {

```

```

00461  /* {1,0,1} = Open Load on OUT1 (f1 must be false; f0&&f1 ruled out above) */
00462  *result_out1 = k_drv8263_olp_open_load;
00463  *result_out2 = k_drv8263_olp_normal;
00464  rx_log_warn(s_tag, "OLP: Open load on OUT1");
00465  } else if (!(int)f0 && (int)f1 && (int)f2) {
00466  /* {0,1,1} = Short to GND */
00467  *result_out1 = k_drv8263_olp_short_to_gnd;
00468  *result_out2 = k_drv8263_olp_short_to_gnd;
00469  rx_log_error(s_tag, "OLP: Short to GND detected");
00470  } else if (!(int)f0 && !(int)f1 && !(int)f2) {
00471  /* {0,0,0} = Short to VM */
00472  *result_out1 = k_drv8263_olp_short_to_vm;
00473  *result_out2 = k_drv8263_olp_short_to_vm;
00474  rx_log_error(s_tag, "OLP: Short to VM detected");
00475  } else {
00476  /* Unexpected pattern */
00477  *result_out1 = k_drv8263_olp_unknown;
00478  *result_out2 = k_drv8263_olp_unknown;
00479  rx_log_warn(s_tag, "OLP: Unknown nFAULT pattern");
00480  }
00481 }
00482
00483 /*
=====
00484 * Public API Implementation
00485 *
=====
*/
00486
00487 /**
00488  * @brief Initialize DRV8263H-Q1 driver instance
00489  *
00490  * @details
00491  * Validates all GPIO assignments in the provided configuration, copies
00492  * the configuration into the handle, and marks the handle as initialized.
00493  * If the configuration flag olp_enable_boot is set, the function
00494  * additionally runs a full Open Load Protection (OLP) diagnostic sequence
00495  * during initialization to detect disconnected or shorted motors before
00496  * the control loop begins. OLP failures at boot are logged as warnings
00497  * but do not prevent initialization from succeeding.
00498  *
00499  * Initialization sequence:
00500  * 1. Validate handle and config pointers (null check)
00501  * 2. Validate all five GPIO port/pin pairs (DRVOFF, nSLEEP, nFAULT, IN1, IN2)
00502  * 3. Copy configuration into handle
00503  * 4. Set handle->initialized = true
00504  * 5. (Optional) Run OLP diagnostic if olp_enable_boot is true
00505  *
00506  *
00507  *
00508  * @pre handle must point to valid, writable memory of size >= sizeof(rx_drv8263_handle_t)
00509  * @pre config must contain valid GPIO port (0-16) and pin (0-7) assignments for all signals
00510  * @pre GPIO ports referenced in config must have their clocks enabled and pin directions set
00511  *
00512  * @post handle->initialized == true on success
00513  * @post handle->config contains a complete copy of *config
00514  * @post On failure, handle is not modified (remains in its prior state)
00515  *
00516  * @note Not thread-safe; do not call concurrently for the same handle
00517  *
00518  * @par Example:
00519  * @code
00520  * static rx_drv8263_handle_t drv;
00521  * const rx_drv8263_config_t cfg = {
00522  *     .port_drvoft = 6, .pin_drvoft = 1,
00523  *     .port_nsleep = 6, .pin_nsleep = 0,
00524  *     .port_nfault = 1, .pin_nfault = 5,
00525  *     .port_in1 = 1, .pin_in1 = 7,
00526  *     .port_in2 = 2, .pin_in2 = 3,
00527  *     .olp_enable_boot = true,
00528  *     .olp_enable_fault = true,
00529  * };
00530  * rx_err_t err = rx_drv8263_init(&drv, &cfg);
00531  * @endcode
00532  *
00533  * @see rx_drv8263_set_drvoft() Control DRVOFF after initialization
00534  * @see rx_drv8263_run_olp() Manual OLP diagnostic
00535  * @see rx_drv8263_clear_latched_fault() Fault reset
00536  *
00537  * @since Version 1.0.0
00538  */
00539 rx_err_t rx_drv8263_init(rx_drv8263_handle_t* handle, const rx_drv8263_config_t* config)
00540 {
00541     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00542     RX_CHECK_NULL_PTR(config, s_tag, "config is nullptr");
00543
00544     const rx_err_t err = internal_validate_config(config);

```



```

00545 if (err != k_rx_ok) {
00546     return err;
00547 }
00548
00549 handle->config = *config;
00550 handle->initialized = true;
00551
00552 rx_log_info(s_tag, "DRV8263H-Q1 driver initialized");
00553
00554 /* Run OLP diagnostic at boot if configured */
00555 if (config->olp_enable_boot) {
00556     rx_drv8263_olp_result_t result_out1 = k_drv8263_olp_unknown;
00557     rx_drv8263_olp_result_t result_out2 = k_drv8263_olp_unknown;
00558
00559     (void)rx_drv8263_run_olp(handle, &result_out1, &result_out2);
00560     bool out1_abnormal = (bool)(result_out1 != k_drv8263_olp_normal);
00561     bool out2_abnormal = (bool)(result_out2 != k_drv8263_olp_normal);
00562     if ((bool)((int)out1_abnormal | (int)out2_abnormal)) {
00563         rx_log_warn(s_tag, "Boot OLP detected abnormal load condition");
00564     }
00565 }
00566
00567 return k_rx_ok;
00568 }
00569
00570 /**
00571  * @brief Control DRVOFF output disable signal
00572  *
00573  * @details
00574  * Drives the DRVOFF GPIO to enable or disable the DRV8263H-Q1 H-bridge
00575  * outputs. DRVOFF is active-high: when active=true, the pin is driven
00576  * HIGH and both half-bridge outputs (OUT1, OUT2) are placed in a
00577  * high-impedance state. When active=false, the pin is driven LOW and
00578  * normal PWM-controlled operation resumes.
00579  *
00580  * DRVOFF must be asserted HIGH before running Open Load Protection (OLP)
00581  * diagnostics, and deasserted LOW to return to normal motor operation.
00582  *
00583  *
00584  *
00585  * @pre handle must be initialized via rx_drv8263_init()
00586  * @pre DRVOFF GPIO must be configured as an output pin (hardware_init responsibility)
00587  *
00588  * @post DRVOFF GPIO reflects the requested level (HIGH if active, LOW otherwise)
00589  * @post No other GPIO pins (nSLEEP, IN1, IN2) are modified
00590  *
00591  * @note Not thread-safe; caller must provide synchronization when accessing
00592  *       the same handle from multiple contexts
00593  *
00594  * @see rx_drv8263_init() Must be called before this function
00595  * @see rx_drv8263_run_olp() Internally uses DRVOFF HIGH for OLP sequence
00596  *
00597  * @since Version 1.0.0
00598  */
00599 rx_err_t rx_drv8263_set_drvoff(rx_drv8263_handle_t* handle, bool active)
00600 {
00601     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00602
00603     if (!handle->initialized) {
00604         rx_log_error(s_tag, "Not initialized");
00605         return k_rx_err_invalid_state;
00606     }
00607
00608     internal_gpio_write(handle->config.port_drvoff, handle->config.pin_drvoff, active);
00609
00610     return k_rx_ok;
00611 }
00612
00613 /**
00614  * @brief Clear latched fault by pulsing nSLEEP LOW
00615  *
00616  * @details
00617  * The DRV8263H-Q1 latches faults (overcurrent, thermal shutdown) until
00618  * the nSLEEP pin is pulsed LOW for a duration within the 5-35 us
00619  * datasheet window. This function executes the following sequence:
00620  *
00621  * 1. Drive nSLEEP LOW to begin the fault reset pulse
00622  * 2. Busy-wait for k_drv8263_nsleep_pulse_us (~20 us), targeting the
00623  *    middle of the 5-35 us specification window
00624  * 3. Drive nSLEEP HIGH to wake the driver
00625  * 4. Wait for tWAKE using tx_thread_sleep(k_twake_threadx_ticks). One
00626  *    ThreadX tick (~10 ms) provides sufficient margin over the ~1.2 ms
00627  *    tWAKE specification. After tWAKE, nFAULT returns HIGH if the
00628  *    underlying fault condition has been removed.
00629  *
00630  * @par Timing Notes:
00631  * The nSLEEP pulse uses a CPU busy-wait loop (internal_delay_us) since

```

```

00632 * the required precision (~20 us) is below ThreadX tick granularity
00633 * (~10 ms). The tWAKE delay uses tx_thread_sleep to yield the CPU.
00634 * nFAULT cannot be polled after the pulse because it may be configured
00635 * as a POEG (Port Output Enable for GPT) input rather than a standard
00636 * GPIO input.
00637 *
00638 *
00639 *
00640 * @pre handle must be initialized via rx_drv8263_init()
00641 * @pre Motor outputs should be stopped (PWM disabled) before clearing a fault
00642 *       to prevent unexpected motor movement when the driver re-enables
00643 *
00644 * @post nSLEEP pin is HIGH (driver awake)
00645 * @post Latched fault is cleared if the underlying fault condition has been removed
00646 * @post At least k_twake_threadx_ticks ThreadX ticks have elapsed (tWAKE satisfied)
00647 *
00648 * @note Not thread-safe; caller must provide synchronization
00649 * @note Cannot verify fault clearance because nFAULT may be a POEG input
00650 *
00651 * @see rx_drv8263_init() Must be called before this function
00652 * @see rx_drv8263_run_olp() Run OLP after clearing fault to diagnose load condition
00653 * @see k_drv8263_nsleep_pulse_us nSLEEP pulse duration constant
00654 * @see k_twake_threadx_ticks ThreadX tick delay for tWAKE
00655 *
00656 * @since Version 1.0.0
00657 */
00658 rx_err_t rx_drv8263_clear_latched_fault(rx_drv8263_handle_t* handle)
00659 {
00660     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00661
00662     if (!handle->initialized) {
00663         rx_log_error(s_tag, "Not initialized");
00664         return k_rx_err_invalid_state;
00665     }
00666
00667     /* Step 1: Drive nSLEEP LOW to begin fault reset pulse */
00668     internal_gpio_write(handle->config.port_nsleep, handle->config.pin_nsleep, false);
00669
00670     /* Step 2: Hold LOW for ~20 us (within 5-35 us spec window) */
00671     internal_delay_us((uint32_t)k_drv8263_nsleep_pulse_us);
00672
00673     /* Step 3: Drive nSLEEP HIGH to wake driver */
00674     internal_gpio_write(handle->config.port_nsleep, handle->config.pin_nsleep, true);
00675
00676     /* Step 4: Wait for tWAKE (~1.2 ms). One ThreadX tick = 10 ms provides
00677      * sufficient margin. Cannot poll nFAULT as it may be configured as POEG input. */
00678     /* NASA Rule 7: capture return value; intentionally discarded (sleep cannot
00679      * meaningfully fail in this context, but we document the discard). */
00680     const UINT sleep_status = tx_thread_sleep((ULONG)k_twake_threadx_ticks);
00681     (void)sleep_status;
00682
00683     rx_log_info(s_tag, "Latched fault cleared");
00684
00685     return k_rx_ok;
00686 }
00687
00688 /**
00689  * @brief Run Open Load Protection (OLP) diagnostic sequence
00690  *
00691  * @details
00692  * Executes the full DRV8263H-Q1 OLP diagnostic to determine the load
00693  * condition on motor outputs OUT1 and OUT2. The sequence temporarily
00694  * places the driver in standby mode (DRVOFF HIGH), applies three IN1/IN2
00695  * test patterns while sampling nFAULT after each, then restores active
00696  * mode (DRVOFF LOW).
00697  *
00698  * Full OLP test sequence:
00699  * 1. Set DRVOFF = HIGH (standby mode, enables OLP detection logic)
00700  * 2. Wait k_drv8263_olp_settle_us (~50 us) for DRVOFF to take effect
00701  * 3. Apply 3 test patterns, each followed by k_drv8263_olp_settle_us delay:
00702  *    - Pattern 0: IN1=0, IN2=0 -> read nFAULT (baseline)
00703  *    - Pattern 1: IN1=1, IN2=0 -> read nFAULT (tests OUT1 path)
00704  *    - Pattern 2: IN1=0, IN2=1 -> read nFAULT (tests OUT2 path)
00705  * 4. Return IN1 and IN2 to LOW (safe state)
00706  * 5. Set DRVOFF = LOW (return to active motor control mode)
00707  * 6. Decode 3-bit nFAULT result vector against the truth table
00708  *
00709  * @par nFAULT Pattern Ordering:
00710  * Patterns are applied in index order (k_olp_pattern_both_low,
00711  * k_olp_pattern_in1_high, k_olp_pattern_in2_high). The nFAULT reading
00712  * for each pattern is stored at the corresponding array index.
00713  *
00714  * @par OLP Truth Table:
00715  * | Pattern 0 | Pattern 1 | Pattern 2 | OUT1 Result | OUT2 Result |
00716  * |-----|-----|-----|-----|-----|
00717  * | 1       | 1       | 1       | Normal     | Normal     |
00718  * | 1       | 1       | 0       | Normal     | Open Load  |

```

```

00719 * | 1      | 0      | 1      | Open Load   | Normal      |
00720 * | 0      | 1      | 1      | Short to GND | Short to GND |
00721 * | 0      | 0      | 0      | Short to VM  | Short to VM  |
00722 * | other   | other   | other   | Unknown     | Unknown     |
00723 *
00724 *
00725 *
00726 * @pre handle must be initialized via rx_drv8263_init()
00727 * @pre Motor PWM must be stopped (IN1/IN2 are driven as GPIO during OLP)
00728 * @pre nFAULT pin must be readable as a standard GPIO input (not solely POEG)
00729 *
00730 * @post DRVOFF returns to LOW (outputs re-enabled)
00731 * @post IN1 and IN2 are both LOW (safe / coast state)
00732 * @post result_out1 and result_out2 contain decoded OLP diagnostic results
00733 *
00734 * @note Not thread-safe; caller must provide synchronization
00735 * @note Execution time: approximately 200 us total (4 x k_drv8263_olp_settle_us
00736 *       settling delays plus GPIO read/write overhead)
00737 *
00738 * @see rx_drv8263_init() Calls this at boot when olp_enable_boot is true
00739 * @see rx_drv8263_set_drvoff() DRVOFF control used internally
00740 * @see internal_olp_apply_patterns() Applies the 3 test patterns
00741 * @see internal_olp_decode_results() Decodes the nFAULT readings
00742 * @see k_drv8263_olp_settle_us Settling delay between patterns
00743 *
00744 * @since Version 1.0.0
00745 */
00746 rx_err_t rx_drv8263_run_olp(rx_drv8263_handle_t* handle,
00747                             rx_drv8263_olp_result_t* result_out1,
00748                             rx_drv8263_olp_result_t* result_out2)
00749 {
00750     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00751     RX_CHECK_NULL_PTR(result_out1, s_tag, "result_out1 is nullptr");
00752     RX_CHECK_NULL_PTR(result_out2, s_tag, "result_out2 is nullptr");
00753
00754     if (!handle->initialized) {
00755         rx_log_error(s_tag, "Not initialized");
00756         return k_rx_err_invalid_state;
00757     }
00758
00759     /* Default results */
00760     *result_out1 = k_drv8263_olp_unknown;
00761     *result_out2 = k_drv8263_olp_unknown;
00762
00763     /* Step 1: Set DRVOFF = HIGH (standby mode, enables OLP detection) */
00764     internal_gpio_write(handle->config.port_drvoff, handle->config.pin_drvoff, true);
00765     internal_delay_us((uint32_t)k_drv8263_olp_settle_us);
00766
00767     /* Step 2: Apply 3 test patterns and read nFAULT for each */
00768     bool nfault_readings[k_drv8263_olp_pattern_count] = {false};
00769     internal_olp_apply_patterns(handle, nfault_readings);
00770
00771     /* Step 3: Return IN1/IN2 to LOW (safe state) */
00772     internal_gpio_write(handle->config.port_in1, handle->config.pin_in1, false);
00773     internal_gpio_write(handle->config.port_in2, handle->config.pin_in2, false);
00774
00775     /* Step 4: Set DRVOFF = LOW (return to active mode) */
00776     internal_gpio_write(handle->config.port_drvoff, handle->config.pin_drvoff, false);
00777
00778     /* Step 5: Decode OLP results from nFAULT readings */
00779     internal_olp_decode_results(nfault_readings[k_olp_pattern_both_low],
00780                                nfault_readings[k_olp_pattern_in1_high],
00781                                nfault_readings[k_olp_pattern_in2_high],
00782                                result_out1,
00783                                result_out2);
00784
00785     return k_rx_ok;
00786 }
00787
00788 /**
00789 * @brief Enable or disable OLP diagnostic at boot time
00790 *
00791 * @details
00792 * Updates the olp_enable_boot flag in the driver configuration stored
00793 * within the handle. When enabled, subsequent calls to rx_drv8263_init()
00794 * with this handle's configuration will execute the full 3-pattern OLP
00795 * diagnostic sequence during initialization. This provides early
00796 * detection of disconnected or shorted motors before the control loop
00797 * begins.
00798 *
00799 * This is a configuration-only change with no immediate hardware side
00800 * effects. The flag takes effect the next time rx_drv8263_init() is
00801 * called with a configuration that has olp_enable_boot set.
00802 *
00803 *
00804 *
00805 * @pre handle must be initialized via rx_drv8263_init()

```

```

00806 * @pre handle must point to valid, writable memory
00807 *
00808 * @post handle->config.olp_enable_boot == enable
00809 * @post No hardware pins are modified (configuration change only)
00810 *
00811 * @note Not thread-safe; caller must provide synchronization when
00812 *       accessing the same handle from multiple contexts
00813 *
00814 * @see rx_drv8263_init() Executes OLP at boot when olp_enable_boot is true
00815 * @see rx_drv8263_set_olp_fault_enable() Companion function for fault-triggered OLP
00816 *
00817 * @since Version 1.0.0
00818 */
00819 rx_err_t rx_drv8263_set_olp_boot_enable(rx_drv8263_handle_t* handle, bool enable)
00820 {
00821     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00822     if (!handle->initialized) {
00823         rx_log_error(s_tag, "Not initialized");
00824         return k_rx_err_invalid_state;
00825     }
00826     handle->config.olp_enable_boot = enable;
00827     return k_rx_ok;
00828 }
00829
00830 /**
00831 * @brief Enable or disable OLP diagnostic in fault handler
00832 *
00833 * @details
00834 * Updates the olp_enable_fault flag in the driver configuration stored
00835 * within the handle. When enabled, the fault handling path will execute
00836 * the full 3-pattern OLP diagnostic sequence after clearing a latched
00837 * fault via rx_drv8263_clear_latched_fault(). This helps determine
00838 * whether the fault was caused by an open or shorted load condition.
00839 *
00840 * This is a configuration-only change with no immediate hardware side
00841 * effects. The flag is checked by the fault handling logic to decide
00842 * whether to run OLP after fault clearance.
00843 *
00844 * @pre handle must be initialized via rx_drv8263_init()
00845 * @pre handle must point to valid, writable memory
00846 *
00847 * @post handle->config.olp_enable_fault == enable
00848 * @post No hardware pins are modified (configuration change only)
00849 *
00850 * @note Not thread-safe; caller must provide synchronization when
00851 *       accessing the same handle from multiple contexts
00852 *
00853 * @see rx_drv8263_clear_latched_fault() Fault clearance function
00854 * @see rx_drv8263_set_olp_boot_enable() Companion function for boot-time OLP
00855 *
00856 * @since Version 1.0.0
00857 */
00858 rx_err_t rx_drv8263_set_olp_fault_enable(rx_drv8263_handle_t* handle, bool enable)
00859 {
00860     RX_CHECK_NULL_PTR(handle, s_tag, "handle is nullptr");
00861     if (!handle->initialized) {
00862         rx_log_error(s_tag, "Not initialized");
00863         return k_rx_err_invalid_state;
00864     }
00865     handle->config.olp_enable_fault = enable;
00866     return k_rx_ok;
00867 }

```